A PRELIMINARY PROJECT REPORT

on

StepCharge AI: Smart Footstep Energy Harvesting and Monitoring System

Submitted to Savitribai Phule Pune University, Pune, in partial fulfilment of the requirements for the award of the degree of

Bachelor of Engineering (Information Technology)

by

[STUDENT NAME 1] **[SEAT NO. 1]**

[STUDENT NAME 2] **[SEAT NO. 2]**

[STUDENT NAME 3] **[SEAT NO. 3]**

[STUDENT NAME 4] **[SEAT NO. 4]**

Under the guidance of

[GUIDE NAME]



Department of Information Technology

MVP'S Karmaveer Baburao Ganpatrao Thakare College of Engineering, Nashik

Nashik–422013

Academic Year 2026–27



MVP'S Karmaveer Baburao Ganpatrao Thakare College of Engineering, Nashik

Nashik–422013

Department of Information Technology

CERTIFICATE

This is to certify that the preliminary project report entitled **StepCharge AI: Smart Footstep Energy Harvesting and Monitoring System** is submitted by the following students in partial fulfilment of the requirements for the project examination of Bachelor of Engineering in Information Technology during the academic year 2026–27.

[STUDENT NAME 1] **[SEAT NO. 1]**

[STUDENT NAME 2] **[SEAT NO. 2]**

[STUDENT NAME 3] **[SEAT NO. 3]**

[STUDENT NAME 4] **[SEAT NO. 4]**

[GUIDE NAME] Project Guide	[COORDINATOR NAME] Project Coordinator
Dr. S. A. Talekar Head of Department	Dr. S. R. Devane Principal

SAVITRIBAI PHULE PUNE UNIVERSITY

CERTIFICATE

This is to certify that,

[STUDENT NAME 1] **[SEAT NO. 1]**

[STUDENT NAME 2] **[SEAT NO. 2]**

[STUDENT NAME 3] **[SEAT NO. 3]**

[STUDENT NAME 4] **[SEAT NO. 4]**

of BE in Information Technology Engineering was

examined in the Project Examination entitled

StepCharge AI: Smart Footstep Energy Harvesting and Monitoring System

on / / 2026

At

Department of Information Technology

MVP'S Karmaveer Baburao Ganpatrao Thakare College of Engineering, Nashik

Nashik–422013

2026–27

Internal Examiner *External Examiner*

Acknowledgement

We express our deepest gratitude to the faculty, management, and administrative leadership of **MVP'S Karmaveer Baburao Ganpatrao Thakare College of Engineering, Nashik**, for providing the necessary infrastructural resources, encouragement, and enriching academic environment that enabled us to undertake this engineering research project.

While the project is actively progressing through its design and experimental phases, we would like to express our profound gratitude to our project guide, [**GUIDE NAME**], for offering invaluable technical direction, constructive criticism, and timely assessments that substantially elevated the academic rigor and practical engineering standards of this work.

We extend our sincere thanks to **Dr. S. A. Talekar**, Head of the Department of Information Technology, for continuous inspiration, academic counseling, and administrative coordination that ensured smooth progress throughout the semester.

We also thank **Dr. S. R. Devane**, Principal, MVP'S Karmaveer Baburao Ganpatrao Thakare College of Engineering, Nashik, for institutional vision and state-of-the-art laboratory infrastructure. Our heartfelt thanks are also due to the project coordinators, the faculty members of the Department of Information Technology, the technical laboratory staff, and the central library personnel for their direct and indirect assistance.

Finally, we convey our appreciation to our peers, families, and colleagues whose collaborative feedback and discussions contributed significantly to the successful execution of this preliminary project phase.

[STUDENT NAME 1]
[STUDENT NAME 2]
[STUDENT NAME 3]
[STUDENT NAME 4]

Abstract

Urban infrastructure and high-density pedestrian facilities generate continuous mechanical kinetic energy that typically dissipates unharnessed into the environment as acoustic vibration and heat. With the proliferation of distributed Internet of Things (IoT) edge nodes, wireless sensor networks, and smart transit tracking beacons, there is a compelling need for decentralized, sustainable micro-energy harvesting coupled with intelligent kinetic analytics. This report presents **StepCharge AI**, an end-to-end intelligent IoT footstep kinetic energy harvesting, storage monitoring, and machine-learning-driven biomechanical analysis system.

The hardware subsystem comprises an array of piezoelectric ceramic transducers (Lead Zirconate Titanate, PZT) embedded within a reinforced floor tile module. Dynamic mechanical compressive stress from human footfalls generates alternating high-voltage, low-current AC electrical pulses via the direct piezoelectric effect. The harvested energy is routed through a low forward-drop Schottky bridge rectifier and transient protection network into a 0.1 F, 5.5 V supercapacitor electrostatic storage buffer. An Espressif ESP32 microcontroller performs edge signal conditioning via 12-bit analog-to-digital converter (ADC) channels with 16× oversampling and Exponential Moving Average (EMA) filtering ($\alpha = 0.25$) on calibrated precision voltage dividers. Footstep impacts are isolated using a dual-threshold Schmitt trigger with hysteresis ($V_{\text{high}} = 0.80\,\text{V}$, $V_{\text{low}} = 0.55\,\text{V}$) and a 200–250 ms refractory lockout window to eliminate false double-triggering.

The edge telemetry pipeline transmits encrypted time-series frames over TLS/HTTPS using SHA-256 authenticated device keys to a cloud-hosted Node.js + Express backend integrated with MongoDB Atlas. Real-time telemetry frames and footstep events are broadcast via Socket.IO room namespaces to an interactive React 19 + TypeScript web dashboard. To transform physical kinetic signals into actionable biomechanical insights, a Python FastAPI microservice extracts five time-domain kinetic features (V_{peak}, V_{avg}, T_{pulse}, ΔT_{step}, V_{storage}) and classifies steps into `LIGHT`, `NORMAL`, and `HEAVY` categories using a Random Forest classifier. To prevent gait identity memorization and overly optimistic performance estimation, model evaluation enforces subject-independent Stratified Group K-Fold cross-validation across participants. Furthermore, a closed-loop load switching architecture ensures that demo loads (LED and low-power DC fan) are visually rendered on the dashboard only upon verified GPIO hardware readback.

Keywords: Energy Harvesting, Piezoelectric Transducers, Internet of Things (IoT),

ESP32, Biomechanical Monitoring, Machine Learning, Random Forest, Supercapacitor Storage, Real-Time Telemetry, MongoDB Atlas.

Abbreviations

Abbreviation	Full Form
ADC	Analog-to-Digital Converter
API	Application Programming Interface
BLE	Bluetooth Low Energy
BOM	Bill of Materials
CI/CD	Continuous Integration / Continuous Deployment
CMOS	Complementary Metal-Oxide-Semiconductor
CPU	Central Processing Unit
CSV	Comma-Separated Values
DFD	Data Flow Diagram
EMA	Exponential Moving Average
EMI	Electromagnetic Interference
ER	Entity–Relationship
ESR	Equivalent Series Resistance
GPIO	General Purpose Input/Output
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
I2C	Inter-Integrated Circuit
IEEE	Institute of Electrical and Electronics Engineers
IoT	Internet of Things
JSON	JavaScript Object Notation
JWT	JSON Web Token
ML	Machine Learning
MOSFET	Metal-Oxide-Semiconductor Field-Effect Transistor
ODM	Object Data Modeling
PZT	Lead Zirconate Titanate ($Pb[Zr_xTi_{1-x}]O_3$)
RAM	Random Access Memory

Abbreviation	Full Form
RBAC	Role-Based Access Control
REST	Representational State Transfer
RF	Radio Frequency
RMS	Root Mean Square
SDLC	Software Development Life Cycle
SHA-256	Secure Hash Algorithm 256-bit
SoC	System on Chip
SPPU	Savitribai Phule Pune University
SRS	Software Requirement Specification
TLS	Transport Layer Security
TVS	Transient Voltage Suppressor
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator
UX	User Experience
WBS	Work Breakdown Structure

Contents

Acknowledgement i

Abstract ii

Abbreviations iv

1 Introduction 1

1.1 Introduction . . . 1
1.2 Project Idea . . . 1
1.3 Motivation of the Project . . . 2
1.4 Background of Footstep Energy Harvesting . . . 3
1.5 Role of IoT in Energy Monitoring . . . 3
1.6 Role of Machine Learning in Footstep Classification . . . 4
1.7 Project Overview . . . 4
1.8 Objectives of the Project . . . 5
1.9 Organization of the Report . . . 6

2 Literature Survey 8

2.1 Introduction . . . 8
2.2 Existing Footstep Energy Harvesting Systems . . . 8
2.3 Piezoelectric Energy Harvesting Technologies . . . 9
2.4 IoT-Based Energy Monitoring Systems . . . 10
2.5 ESP32-Based Monitoring Systems . . . 10
2.6 Machine Learning-Based Footstep Classification . . . 11
2.7 Existing Research Approaches . . . 12
2.8 Comparative Analysis . . . 12
2.9 Research Gaps . . . 14
2.10 Conclusion of Literature Survey . . . 14

3 Problem Definition and Scope 16

3.1 Problem Statement . . . 16
3.2 Goals and Objectives . . . 17
3.3 Statement of Scope . . . 18

3.4 Existing System . . . 19
3.5 Limitations of Existing Approaches . . . 19
3.6 Proposed System . . . 20
3.7 Software Context . . . 20
3.8 Major Constraints . . . 21
3.9 Methodologies of Problem Solving and Efficiency Issues . . . 22
3.10 Scenario in Which Multi-Core, Embedded and Distributed Computing Used . . . 22
3.11 Expected Outcomes . . . 23
3.12 Applications . . . 23
3.13 Hardware Resources Required . . . 24
3.14 Software Resources Required . . . 24

4 Software Requirement Specification 26
4.1 Introduction . . . 26
4.1.1 Purpose and Scope of the Document . . . 26
4.1.2 Overview of Developer Responsibilities . . . 26
4.2 Functional Requirements . . . 27
4.2.1 Footstep Detection and Hysteresis Thresholding (FR-1) . . . 27
4.2.2 Dual-Channel Voltage Measurement and Digital Filtering (FR-2) 27
4.2.3 Energy Storage Monitoring and Safety Thresholds (FR-3) . . . 28
4.2.4 Real-Time Machine Learning Inference and Classification (FR-4) 28
4.2.5 ESP32 Telemetry Transmission and Reconnection Buffer (FR-5) 28
4.2.6 User Authentication and Role-Based Access Control (FR-6) . . . 29
4.2.7 Real-Time Dashboard Observability (FR-7) . . . 29
4.2.8 Historical Telemetry Logging and Export (FR-8) . . . 29
4.2.9 Closed-Loop Load Actuation and Feedback Verification (FR-9) 30
4.2.10 Automated System Health and Anomaly Alerts (FR-10) . . . 30
4.3 Non-Functional Requirements . . . 30
4.3.1 Performance Requirements . . . 30
4.3.2 Reliability and Fault Tolerance . . . 31
4.3.3 Security Requirements . . . 31
4.3.4 Availability . . . 32
4.3.5 Scalability . . . 32
4.3.6 Usability . . . 32
4.3.7 Maintainability . . . 32
4.4 Hardware Requirements . . . 32
4.5 Software Requirements . . . 33
4.6 Database Requirements . . . 33

4.7 System Constraints . . . 34
4.8 Development Methodology: The Agile SDLC Model . . . 34
4.8.1 Justification of the Agile Methodology . . . 34
4.8.2 Implementation Phases in Agile . . . 36

5 System Design **37**
5.1 System Architecture . . . 37
5.2 Overall Hardware Architecture . . . 38
5.3 Piezoelectric Energy Harvesting Subsystem . . . 38
5.4 Rectifier and Power Conditioning Circuit . . . 38
5.5 Supercapacitor Energy Storage . . . 39
5.6 Voltage Measurement Subsystem . . . 39
5.7 ESP32 Embedded Monitoring and Signal Acquisition . . . 40
5.8 IoT Communication Architecture . . . 40
5.9 Backend Architecture . . . 40
5.10 Database Design . . . 40
5.11 Machine Learning Architecture . . . 41
5.12 Frontend Dashboard Architecture . . . 41
5.13 Data Flow Diagrams . . . 41
5.13.1 Data Flow Diagram Level 0 (Context Level) . . . 41
5.13.2 Data Flow Diagram Level 1 . . . 42
5.14 Entity Relationship Diagram . . . 42
5.15 Unified Modeling Language (UML) Diagrams . . . 42
5.15.1 Use Case Diagram . . . 43
5.15.2 Activity Diagram . . . 43
5.15.3 State Machine Diagram . . . 44
5.15.4 Class Diagram . . . 45
5.15.5 Sequence Diagram . . . 45
5.15.6 Object Diagram . . . 46

6 Implementation and Technical Specifications **47**
6.1 Introduction . . . 47
6.2 Hardware Implementation and Prototyping . . . 47
6.3 Piezoelectric Sensor Integration . . . 47
6.4 Rectification, Conditioning and Supercapacitor Storage . . . 48
6.5 ESP32 Firmware Implementation . . . 48
6.6 Sensor Data Acquisition and Calibration . . . 48
6.7 Wi-Fi Communication and Secure REST Telemetry . . . 49
6.8 Backend API Implementation . . . 49

6.9 Google Authentication and Role-Based Access Control 50
6.10 MongoDB Atlas Integration and Time-Series Optimization 50
6.11 Real-Time Communication Using Socket.IO 51
6.12 Machine Learning Implementation 51
6.13 Data Preprocessing and Feature Extraction 51
6.14 Footstep Classification and Subject-Independent Validation 52
6.15 React Dashboard Implementation 52
6.16 Integration of Hardware and Software 52
6.17 Security and Error Handling 53

7 Testing and Results **54**
7.1 Introduction 54
7.2 Hardware Testing 54
7.3 Voltage Measurement Testing 54
7.4 Footstep Detection Testing 55
7.5 ESP32 Communication Testing 55
7.6 Backend API and Security Testing 56
7.7 Database Testing 57
7.8 Machine Learning Model Evaluation 57
7.8.1 Subject-Independent vs. Random Split Evaluation 57
7.8.2 Confusion Matrix and Classification Metrics 58
7.9 Dashboard Testing 58
7.10 Integration Testing 59
7.11 Comprehensive Test Case Table 59
7.12 Observed Results 61
7.13 Result Analysis 61
7.14 Limitations 62

8 Other Specifications **63**
8.1 Advantages 63
8.2 Limitations 64
8.3 Applications 64
8.4 Future Scope 65
8.5 Sustainability and Environmental Benefits 65
8.6 Cost Considerations and Bill of Materials 66

9 Project Plan **68**
9.1 Project Implementation Plan 68
9.2 Work Breakdown Structure (WBS) 69
9.3 Gantt Chart 70

9.4 Semester-Wise Activities 70
9.5 Work Completed 71
9.6 Pending Work 71

10 Conclusion and Future Work **72**
10.1 Conclusion 72
10.2 Future Enhancements 73

References **74**

A Hardware Specifications and Pinout Mapping **75**
A.1 ESP32 GPIO Pinout Configuration 75
A.2 Transducer Matrix Electrical Properties 76
A.3 Power Conditioning Component Parameters 76

B Important Code Snippets **77**
B.1 ESP32 Firmware Step Detection Loop 77
B.2 Node.js Device Telemetry Controller 78
B.3 Machine Learning Feature Extractor and Trainer 78

C System Photographs and Dashboard Visuals **80**
C.1 Smart Footstep Energy Harvester Prototype 80
C.2 Hardware and Circuit Instrumentation 80
C.3 Real-Time Kinetic Energy Flow Monitoring 81
C.4 Machine Learning Classification Visuals 81
C.5 IoT Network and Telemetry Architecture 82

Annexure A: Plagiarism Report **83**

Annexure B: Activities Completed During Semester 1 **84**

List of Figures

4.1 The iterative Agile Software Development Life Cycle (SDLC) model applied to StepCharge AI. 35

5.1 Overall StepCharge AI multi-tier cyber-physical system architecture. 37
5.2 Detailed hardware block diagram of the StepCharge AI edge harvesting node. 38
5.3 Electrical schematic of rectification, clamping, supercapacitor, and dual-divider sensing. 39
5.4 Schmitt-trigger hysteresis detection waveform and refractory lockout window. 40
5.5 Decoupled Machine Learning architecture: offline validation and online real-time inference. 41
5.6 Data Flow Diagram (DFD) Level 0 — Context Level Diagram. 41
5.7 Data Flow Diagram (DFD) Level 1 — Functional Process Decomposition. 42
5.8 Entity-Relationship (ER) diagram representing the MongoDB Atlas schema. 42
5.9 Use Case Diagram modeling user and physical actor interactions. . . 43
5.10 Activity Diagram illustrating the edge firmware step detection and telemetry cycle. 44
5.11 State Machine Diagram of the ESP32 edge firmware. 44
5.12 Class Diagram illustrating primary backend models and services. . . . 45
5.13 Sequence Diagram of footstep acquisition, ML classification, and UI update. 45
5.14 Object Diagram capturing runtime instances during a footstep event. 46

7.1 Empirical supercapacitor voltage charging curve over 60 consecutive footsteps. 61

9.1 Gantt chart illustrating project development phases across Academic Year 2026–27. 70

C.1 StepCharge AI physical footstep energy harvesting floor tile prototype. 80

C.2 Internal hardware layout showing PZT transducer array and ESP32 conditioning circuitry. 81
C.3 Real-time energy flow, voltage accumulation, and load management interface. 81
C.4 Machine learning footstep classification and kinetic intensity feed. . . 82
C.5 IoT edge-to-cloud communication and real-time streaming pipeline. . 82

List of Tables

2.1 Comprehensive comparative analysis of footstep energy harvesting and monitoring systems in published literature versus StepCharge AI. . . 13

3.1 Hardware specifications and prototype bill of materials. 24
3.2 Software frameworks, development environments, and toolchain. . . . 25

7.1 Voltage measurement calibration and ADC conversion accuracy across operational range. 55
7.2 Confusion matrix under subject-independent Stratified Group K-Fold validation. 58
7.3 Detailed Machine Learning classification metrics per gait class. 58
7.4 Comprehensive system test cases, empirical verification, and validation status. 60

8.1 Comprehensive prototype Bill of Materials (BOM) and cost breakdown. 67

9.1 System implementation plan and milestone schedule. 68

A.1 ESP32 hardware GPIO pin assignment and peripheral configuration. 75

Chapter 1

Introduction

1.1 Introduction

Rapid urbanization, smart city modernization, and the explosive proliferation of distributed Internet of Things (IoT) devices have intensified the demand for localized, sustainable, and maintenance-free auxiliary power generation. Conventional edge sensor nodes, structural health monitors, and public transit tracking beacons rely predominantly on chemical batteries or hardwired grid infrastructure. Chemical batteries, however, introduce substantial maintenance overheads, finite operational lifespans, periodic replacement costs, and severe environmental recycling concerns. Conversely, human locomotion in high-density facilities—such as metropolitan railway terminals, airport concourses, university campuses, shopping complexes, and industrial walkways—continuously deposits substantial mechanical kinetic energy into floor surfaces. In standard architectural environments, this kinetic impact energy dissipates completely as acoustic noise and low-grade thermal vibration into the flooring substrate.

Energy harvesting, also referred to as ambient power scavenging, offers an elegant paradigm to reclaim this otherwise wasted kinetic energy. By utilizing piezoelectric ceramic materials that exhibit the direct piezoelectric effect, mechanical compressive stress generated during human heel-strike and toe-off transitions can be converted directly into electrical charge. When paired with high-efficiency rectification, capacitive energy accumulation, low-power edge microcontrollers, and modern cloud analytics, footstep energy harvesting transforms inert architectural surfaces into active, decentralized micro-power stations and intelligent human activity monitoring platforms.

1.2 Project Idea

The project idea behind **StepCharge AI: Smart Footstep Energy Harvesting and Monitoring System** is to develop an integrated cyber-physical platform that bridges physical energy harvesting and digital data intelligence. Rather than treating kinetic energy harvesting merely as an isolated electrical power exercise, StepCharge AI instruments the physical tile with edge sensing, high-speed telemetry, cloud databases, machine learning microservices, and real-time visualization dash-

boards.

The physical tile comprises a parallel-series array of Lead Zirconate Titanate (PZT) piezoelectric ceramic discs housed within an impact-absorbing mechanical housing. Mechanical footfalls produce transient high-voltage alternating current (AC) pulses, which are routed through a low forward-drop Schottky bridge rectifier and buffered into a 0.1 F, 5.5 V supercapacitor. Simultaneously, an Espressif ESP32 dual-core microcontroller samples the rectified pulse and storage rail voltages using calibrated analog-to-digital converter (ADC) channels. The edge firmware applies real-time digital filtering, executes dual-threshold Schmitt trigger step detection, and transmits telemetry frames over Wi-Fi via secure TLS/HTTPS REST APIs to a Node.js and Express backend. The data is persisted in MongoDB Atlas, streamed instantaneously via Socket.IO to a React web dashboard, and analyzed by a Python machine learning microservice that extracts kinetic feature vectors to classify footstep intensity into light, normal, and heavy walking profiles.

1.3 Motivation of the Project

The primary motivations driving the development of StepCharge AI include:

1. **Decentralized Green Energy for Edge Devices:** Eliminating or reducing dependence on disposable electrochemical batteries for localized low-power IoT devices (such as BLE beacons, pathway lighting, and environmental sensor nodes) by capturing clean, renewable kinetic energy directly from human movement.

2. **Lack of Data Instrumentation in Conventional Harvesters:** Most existing academic and hobbyist footstep harvesting setups operate strictly as unmonitored analog power circuits. They lack continuous digital measurement, data persistence, and remote telemetry, making it difficult to assess real-world energy yields, temporal degradation, or usage patterns.

3. **Synergy between Energy Scavenging and Biomechanical Sensing:** Every footstep produces a distinctive voltage-time transient wave that encodes physical impact force, stance duration, and cadence. This presents an unexploited opportunity to combine energy harvesting with non-intrusive gait monitoring, foot traffic density counting, and intensity classification using artificial intelligence.

4. **Rigorous Engineering Provenance:** Much of the existing literature claims unrealistic power outputs without proper measurement transparency. There

is a pressing need for an honest, research-grade engineering implementation that explicitly distinguishes directly measured physical quantities from derived estimates, while upholding strict scientific evaluation standards.

1.4 Background of Footstep Energy Harvesting

The physical foundation of kinetic footstep energy scavenging lies in the direct piezoelectric effect discovered by Jacques and Pierre Curie in 1880. Certain anisotropic crystalline and polycrystalline ceramic materials—most notably Lead Zirconate Titanate ($Pb[Zr_xTi_{1-x}]O_3$, or PZT)—exhibit electric dipole polarization when subjected to external mechanical strain. The constitutive electromechanical relationship governing the direct piezoelectric effect is expressed as:

$$D_i = d_{ijk}\sigma_{jk} + \varepsilon_{ik}^{T} E_k \tag{1.1}$$

where D_i denotes the electric displacement vector (C/m^2), d_{ijk} represents the piezoelectric charge coefficient tensor (C/N), σ_{jk} is the applied mechanical stress tensor (N/m^2), ε_{ik}^{T} is the dielectric permittivity at constant mechanical stress (F/m), and E_k is the internal electric field vector (V/m).

When a pedestrian steps onto a piezoelectric tile, the applied vertical impact generates a rapid stress transient $\sigma(t)$, generating positive and negative charges on the opposing metallized electrodes of the disc. Because human gait produces oscillatory loading and unloading cycles, the resulting electrical output is inherently alternating current (AC) with high open-circuit peak voltages (often 10 V to 40 V depending on impact velocity and disc quantity) but high internal source impedance, yielding microampere-to-milliampere-level currents. To render this electrical energy usable for direct current (DC) electronics, the raw waveform must undergo full-wave rectification and capacitive smoothing before transfer into an electrostatic storage buffer, such as an electric double-layer supercapacitor.

1.5 Role of IoT in Energy Monitoring

The Internet of Things (IoT) serves as the digital backbone that elevates StepCharge AI from a passive energy collector to a connected edge system. An instrumented IoT architecture introduces the following essential capabilities:

- **Edge Signal Acquisition:** High-speed sampling of fluctuating analog voltages directly at the point of generation, executing local noise rejection without loading down the high-impedance transducer output.

- **Real-Time Wireless Telemetry:** Transmitting instantaneous voltage levels, step detection events, cumulative step counts, and hardware health metrics over standard IEEE 802.11 b/g/n Wi-Fi channels.
- **Cloud Persistence and Scalability:** Ingesting time-series telemetry into high-availability cloud database clusters (MongoDB Atlas), enabling multi-device aggregation, historical rollups, and longitudinal performance tracking.
- **Bidirectional Control and Telecommand:** Providing a secure control channel to dispatch commands to edge devices, such as toggling demonstration loads (LEDs, DC fans) and verifying real GPIO readback states.

1.6 Role of Machine Learning in Footstep Classification

While traditional footstep counters rely solely on fixed hardware comparators or simple threshold crossings, human gait dynamics are complex, multi-dimensional, and variable across individuals. Walking cadence, footwear compliance, body mass, and stepping force modulate the kinetic pulse waveform in distinct ways. A hard impact by a running individual produces a sharp rise time with elevated peak voltage, whereas a gentle, shuffling step produces a flatter, elongated pulse with moderate amplitude.

Machine learning enables automated, multi-feature pattern recognition on these kinetic transients. By extracting deterministic physical feature vectors—specifically peak pulse voltage (V_{peak}), mean voltage over pulse duration (V_{avg}), pulse duration (T_{pulse}), inter-step cadence interval (ΔT_{step}), and buffer rail storage voltage (V_{storage})—supervised classification algorithms such as Random Forests can categorize footsteps into distinct intensity classes (`LIGHT`, `NORMAL`, `HEAVY`). This classification provides valuable behavioral analytics for smart floor installations, identifying pedestrian congestion zones, walking speed distributions, and dynamic foot-traffic loading without invading individual privacy through optical cameras or intrusive wearable sensors.

1.7 Project Overview

StepCharge AI integrates multiple engineering disciplines into a cohesive six-tier architecture:

1. **Edge Transducer and Power Conditioning Layer:** Housed within a custom mechanical tile structure, consisting of piezoelectric discs, full-wave Schottky bridge rectification, TVS overvoltage clamping, and a 0.1 F / 5.5 V supercapacitor storage reservoir.

2. **Embedded Microcontroller Layer:** An ESP32 microcontroller operating custom C++ firmware that executes 50 Hz analog envelope acquisition, 16× ADC oversampling, Exponential Moving Average smoothing, Schmitt-trigger step detection, and TLS-encrypted REST transmission.

3. **Backend Application Layer:** A Node.js and Express server written in TypeScript providing RESTful API endpoints, device authentication via cryptographic SHA-256 tokens, Google OAuth 2.0 user sessions with HTTP-only cookies, and rate-limiting security guards.

4. **Real-Time Streaming Layer:** An integrated Socket.IO gateway managing device-scoped room subscriptions to push instantaneous telemetry and event notifications to active clients with sub-100 ms latency.

5. **Database Layer:** MongoDB Atlas document database structured with optimized compound indices on device identifiers and descending timestamps, supporting sub-millisecond range queries and historical aggregation pipelines.

6. **Machine Learning Microservice Layer:** A Python FastAPI service hosting a scikit-learn Random Forest model trained using subject-independent Stratified Group K-Fold cross-validation to classify footstep impacts in real time.

7. **Client Dashboard Layer:** A modern, responsive web application engineered in React 19, TypeScript, Vite, and Tailwind CSS, featuring live telemetry dials, interactive time-series charts, AI classification feeds, historical data tables, and closed-loop load switching indicators.

1.8 Objectives of the Project

The primary technical and academic objectives of StepCharge AI are formulated as follows:

1. To design, construct, and calibrate a physical piezoelectric energy harvesting floor tile prototype capable of generating measurable electrical energy from standard pedestrian footsteps.

2. To implement a power conditioning circuit incorporating Schottky bridge rectification and a 0.1 F supercapacitor storage buffer with hardware and software overvoltage protection.

3. To develop edge firmware for the ESP32 microcontroller that reliably samples analog voltages, applies digital filtering, detects steps via a dual-threshold

Schmitt trigger with hysteresis, and rejects false double-triggers through refractory lockout timing.

4. To construct an asynchronous, secure REST API using Node.js, Express, and TypeScript that authenticates edge devices using SHA-256 tokens and enforces role-based access control (RBAC).
5. To persist edge telemetry and discrete footstep events into a cloud-hosted MongoDB Atlas database utilizing time-series indexing.
6. To implement a real-time event distribution engine using Socket.IO that streams live metrics to web clients without continuous client-side polling.
7. To build and evaluate a supervised Machine Learning pipeline in Python that extracts time-domain physical features from footstep transients and classifies impacts into intensity categories using subject-independent validation.
8. To engineer a web-based dashboard using React 19, TypeScript, and Tailwind CSS that renders real-time dials, historical charts, AI prediction confidence, and closed-loop load confirmation.
9. To uphold strict engineering measurement honesty by maintaining clear distinctions between measured electrical quantities and calculated capacitor energy estimates.

1.9 Organization of the Report

The remainder of this report is organized into the following chapters:

- **Chapter 2 (Literature Survey):** Reviews published research in kinetic energy harvesting, piezoelectric transducers, IoT energy monitoring, and machine-learning-based gait classification, presenting a comparative analysis table and identifying key research gaps.
- **Chapter 3 (Problem Definition and Scope):** Formulates the engineering problem statement, defines project scope and constraints, details problem-solving methodologies, and outlines required hardware and software resources.
- **Chapter 4 (Software Requirement Specification):** Specifies functional and non-functional requirements, database schemas, system interfaces, and justifies the adoption of the Agile SDLC model.
- **Chapter 5 (System Design):** Presents the complete architectural design, hardware block diagrams, circuit schematics, Data Flow Diagrams (Level 0 and

Level 1), Entity-Relationship diagrams, and UML diagrams (Use Case, Activity, State, Sequence, Class, and Object).

- **Chapter 6 (Implementation and Technical Specifications):** Details the practical implementation of the hardware assembly, ESP32 firmware algorithms, backend REST controllers, Google OAuth, Socket.IO channels, ML feature extraction, and React dashboard components.
- **Chapter 7 (Testing and Results):** Documents unit, integration, and security test cases (including the 101 Vitest backend test suite), presents experimental hardware waveforms, evaluates ML classification performance, and analyzes empirical results.
- **Chapter 8 (Other Specifications):** Explores advantages, physical limitations, practical applications, future enhancements, sustainability impacts, and cost considerations.
- **Chapter 9 (Project Plan):** Outlines the semester-wise implementation plan, Work Breakdown Structure, Gantt chart, completed milestones, and pending work for Semester 2.
- **Conclusions and References:** Synthesizes the project findings and provides IEEE-formatted references and technical appendices.

Chapter 2

Literature Survey

2.1 Introduction

Kinetic energy harvesting from human locomotion and ambient mechanical vibrations has garnered extensive academic and industrial interest over the past two decades. The convergence of low-power embedded microcontrollers, wireless communication protocols, and machine learning has expanded the focus from purely electrical power extraction to intelligent cyber-physical systems. This chapter reviews foundational and contemporary research in piezoelectric transduction, power conditioning, Internet of Things (IoT) energy monitoring architectures, and machine-learning-based footstep classification. Furthermore, this survey provides a structured comparative analysis of existing systems, explicitly articulates prevailing research gaps, and establishes the academic rationale for the proposed StepCharge AI architecture.

2.2 Existing Footstep Energy Harvesting Systems

Early explorations into biomechanical energy harvesting emerged in the late 1990s and early 2000s, driven by the vision of autonomous wearable computing. Kymissis et al. [1] conducted pioneering work on parasitic power harvesting in footwear at the MIT Media Laboratory. Their investigation evaluated three distinct transducer mechanisms: a flexible polyvinylidene fluoride (PVDF) piezoelectric foil stave in the insole, a rigid Lead Zirconate Titanate (PZT) unimorph transducer positioned in the heel, and a miniature rotary electromagnetic generator coupled to heel-strike mechanics. The PZT heel transducer generated up to 80 mJ per step at open-circuit peak potentials approaching 60 V. However, the mechanical rigidity of PZT ceramics raised reliability concerns regarding fatigue fractures under repeated dynamic impacts, while the PVDF stave, although flexible and mechanically compliant, yielded significantly lower average power (approximately 1 mW).

Building upon these findings, Paradiso and Starner [2] synthesized energy scavenging techniques across mobile and pervasive computing environments. They categorized kinetic harvesting mechanisms into passive parasitic harvesters (which harness secondary energy dissipated during standard locomotion) and active user-driven devices. Their analysis revealed that while normal walking expends approximately 67 W

of biomechanical power in heel-strike decelerations, extracting even 1% of this energy without causing user discomfort requires careful mechanical impedance matching and compliance optimization.

Subsequent research shifted toward stationary pedestrian infrastructure. Instead of integrating harvesters into individual shoes, researchers embedded piezoelectric transducer arrays beneath floor tiles in high-traffic corridors. Sravani et al. [3] designed and evaluated an instrumented piezoelectric flooring tile incorporating circular PZT diaphragm discs. Their prototype demonstrated that mechanical force distribution across a multi-transducer matrix could generate sufficient instantaneous electrical charge to illuminate low-power signaling LEDs. However, their setup remained a self-contained analog circuit lacking digital measurement instrumentation, wireless data logging, or analytical intelligence.

2.3 Piezoelectric Energy Harvesting Technologies

Piezoelectric materials operate via electromechanical coupling between mechanical strain and electrical charge displacement. In kinetic harvesting applications, two broad material classes predominate:

1. **Piezoceramics (Lead Zirconate Titanate, PZT):** PZT materials exhibit high piezoelectric charge coefficients ($d_{33} \approx 300$–$600\,\mathrm{pC/N}$) and high electromechanical coupling factors ($k_{33} \approx 0.70$). Consequently, they generate substantial electrical energy density under mechanical compression. However, piezoceramics are inherently brittle, necessitating mechanical protection, shock-absorbing elastomers, and load-distribution plates to prevent cracking under excessive vertical impact stresses.

2. **Piezopolymers (Polyvinylidene Fluoride, PVDF):** PVDF and its copolymers offer exceptional mechanical flexibility, high mechanical strength, and fatigue resistance under cyclic deformation. Nevertheless, their piezoelectric charge coefficient is substantially lower ($d_{33} \approx -20$– $-35\,\mathrm{pC/N}$), resulting in energy generation that is typically an order of magnitude lower than PZT for equivalent impact forces.

Erturk and Inman [4] provided rigorous electroelastic modeling of piezoelectric energy harvesters, demonstrating that the electrical power delivered to a load depends critically on the electrical source impedance matching the load impedance at the excitation frequency. Because human footsteps occur at low fundamental frequencies

(1.0–2.5 Hz), the internal capacitive impedance of a piezoelectric disc:

$$Z_C = \frac{1}{2\pi f C_p} \tag{2.1}$$

is exceedingly high (often 100 kΩ to several MΩ, where C_p is the internal transducer capacitance, typically 10–50 nF). Consequently, direct connection of a low-impedance electrical load severely collapses the output voltage, yielding negligible power transfer. This fundamental physical constraint mandates an intermediate energy conditioning stage comprising rectification and capacitive buffering.

Sodano, Inman, and Park [5] investigated power harvesting circuit topologies, comparing standard full-wave diode bridge rectifiers against active synchronized switch harvesting on inductor (SSHI) techniques. While SSHI topologies improve energy extraction efficiency under harmonic vibration, passive bridge rectifiers utilizing low forward-drop Schottky diodes remain the standard for impulsive, non-periodic footstep transients due to their zero quiescent power draw and circuit simplicity.

2.4 IoT-Based Energy Monitoring Systems

The integration of Internet of Things (IoT) frameworks into renewable energy systems has transformed physical power collectors into intelligent telemetry nodes. Al-Fuqaha et al. [6] provided a comprehensive survey of IoT communication protocols, architectures, and data management paradigms. They emphasized the critical role of lightweight application layer protocols (HTTP/REST, MQTT, CoAP, and WebSockets) in transmitting high-frequency sensor updates from resource-constrained microcontrollers to cloud infrastructure.

Shen et al. [7] implemented an edge-assisted intelligent IoT architecture for smart building energy monitoring. Their system demonstrated that shifting preliminary filtering, thresholding, and state detection to edge microcontrollers significantly conserves network bandwidth and reduces cloud processing latency. Rather than streaming raw analog waveforms continuously, the edge device transmits discrete event payloads and periodic statistical aggregations, preserving battery life and maintaining communication robustness under fluctuating network conditions.

2.5 ESP32-Based Monitoring Systems

The Espressif ESP32 system-on-chip (SoC) has emerged as a preferred edge computing platform for IoT instrumentation due to its dual-core Xtensa 32-bit LX6 microprocessor operating up to 240 MHz, integrated 2.4 GHz Wi-Fi (802.11 b/g/n) and

Bluetooth v4.2/BLE, and rich peripheral set [8]. The SoC incorporates two 12-bit Successive Approximation Register (SAR) ADCs supporting up to 18 channels.

However, academic literature identifies key operational constraints regarding the ESP32 ADC:

- **Non-Linear Response Characteristics:** The native ESP32 ADC exhibits non-linear response curves near its operational boundaries (specifically below 0.1 V and above 3.1 V under 11 dB attenuation). Accurate measurement mandates calibrated two-point piecewise characterization and external precision voltage dividing.
- **High-Frequency RF Noise Coupling:** Active Wi-Fi transmissions introduce transient electrical switching noise into the analog subsystem. Mitigating this noise requires multi-sample oversampling (such as 16× hardware accumulation) combined with digital low-pass filtering, such as Exponential Moving Average (EMA) smoothing:

$$y[k] = \alpha \cdot x[k] + (1 - \alpha) \cdot y[k - 1] \tag{2.2}$$

  where $\alpha \in (0, 1]$ represents the smoothing factor.
- **Channel Multiplexing Restrictions:** ADC2 channels cannot be reliably utilized while Wi-Fi transmission is active because the Wi-Fi driver shares internal hardware resources with ADC2. Consequently, precision analog inputs must be routed strictly through ADC1 channels (such as GPIO 34 and GPIO 35).

2.6 Machine Learning-Based Footstep Classification

Biomechanical analysis of human gait traditionally relies on multi-axis force plates, optical motion capture suites, and wearable inertial measurement units (IMUs). Chen et al. [9] conducted an extensive review of biomechanical energy harvesting and gait classification. They highlighted that floor-embedded ground reaction force sensors capture unique temporal force curves characterized by distinct phases: initial heel-strike shock impact, mid-stance weight transfer, and terminal toe-off impulse.

Pedregosa et al. [10] documented the algorithmic implementations of modern machine learning techniques within Python's Scikit-learn framework. For time-series sensor classification, supervised ensemble methods have consistently demonstrated superior generalization and noise tolerance compared to individual decision trees or distance-based classifiers. Breiman's Random Forest algorithm [11] constructs an

ensemble of randomized, de-correlated decision trees:

$$\hat{f}(x) = \frac{1}{B} \sum_{b=1}^{B} f_b(x) \tag{2.3}$$

where each tree f_b is trained on a bootstrap sample of the training set with randomized feature subset splits. Random Forests provide intrinsic out-of-bag error estimation, robust handling of non-linear feature interactions, and computational efficiency suitable for sub-millisecond cloud inferencing.

2.7 Existing Research Approaches

Table 2.1 synthesizes key experimental approaches reported in the literature and contrasts them directly with the proposed StepCharge AI system across hardware, telemetry, machine learning, and measurement methodology.

2.8 Comparative Analysis

Table 2.1: Comprehensive comparative analysis of footstep energy harvesting and monitoring systems in published literature versus StepCharge AI.

Study / Reference	Transducer Matrix	Storage / Yield	Telemetry / IoT Tier	Machine Learning	Key Technical Limitations
Kymissis et al. (1998) [1]	PZT ceramic & PVDF foil in shoe insole	Electrolytic capacitor, $\approx 80\,\text{mJ}$ peak per stride	None (purely analog load circuit)	None	Ceramic element fragility under repetitive mechanical shear; no telemetry, data logging, or digital instrumentation.
Paradiso & Starner (2005) [2]	Survey of piezoelectric & electromagnetic scavengers	Various micro-scale storage buffers	None (isolated wireless beacons only)	None	Theoretical paradigm survey; absence of an integrated, end-to-end cyber-physical IoT architecture.
Sodano et al. (2004) [5]	Unimorph PZT cantilever beam	Rechargeable NiMH button cell	None	None	Benchtop evaluation on mechanical shaker tables; not adapted to dynamic distributed human footsteps.
Sravani et al. (2020) [3]	PZT ceramic discs in tile configuration	Direct bridge to LED output	None	None	Lacks energy storage buffer and microcontroller; zero telemetry or analytical intelligence.
Shen et al. (2018) [7]	Current transformer & environmental sensors	Grid-powered IoT nodes	Edge Gateway / Cloud API	Statistical time-series anomaly detection	Monitors conventional commercial building power grids; possesses no kinetic energy harvesting capability.
Chen et al. (2019) [9]	Wearable IMUs & biometric force plates	External lithium-polymer battery pack	Bluetooth Low Energy to PC workstation	SVM and k-NN gait classification	Highly intrusive wearable sensors requiring patient compliance; no self-powered kinetic scavenging.
StepCharge AI (Proposed)	**8× PZT-5 Ceramic Brass Diaphragm Discs**	**0.1 F / 5.5 V Supercap; dual resistive dividers**	**ESP32 dual-core Wi-Fi, Node.js REST, MongoDB, Socket.IO**	**Random Forest (100 trees), Group K-Fold CV, 5 temporal features**	**Resolves all gaps: direct voltage sensing, closed-loop GPIO readback, and strict physical provenance.**

2.9 Research Gaps

A critical analysis of the surveyed literature reveals five prominent research gaps:

1. **Fragmentation between Energy Harvesting and Digital Instrumentation:** Existing literature is sharply bifurcated into two mutually exclusive domains: physical electrical engineering research that treats tiles as isolated analog circuits without telemetry, and computer science research that deploys IoT sensor dashboards powered by conventional batteries. A unified, fully integrated cyber-physical architecture combining kinetic scavenging, digital filtering, and real-time cloud data pipelines remains conspicuously absent.

2. **Inflation of Harvested Energy and Lack of Provenance Transparency:** Numerous published studies claim unrealistic power outputs by multiplying peak open-circuit voltage spikes by theoretical short-circuit currents without matching source impedance or accounting for capacitor leakage, diode forward-voltage drops, and internal ESR. There is a marked deficiency in research that clearly differentiates directly measured quantities from calculated capacitor energy estimates.

3. **Methodological Flaws in Machine Learning Gait Evaluation:** Existing studies applying machine learning to footstep and gait data predominantly employ standard random train/test splits. When samples from the same individual appear in both training and test partitions, the classifier memorizes idiosyncratic personal cadence and gait rhythms rather than generalizing to kinetic impact intensity. Subject-independent cross-validation protocols are rarely enforced.

4. **Absence of Closed-Loop Load Actuation Feedback:** IoT energy platforms with remote load toggling frequently adopt open-loop command architectures, assuming that dispatching a network command implies physical actuation. They lack hardware readback mechanisms to verify actual GPIO logic states.

5. **Omission of Production-Grade Cloud Security:** Academic IoT prototypes routinely embed raw database credentials and unencrypted keys within firmware, bypass transport-layer encryption, and omit user authorization mechanisms (such as RBAC and HTTP-only session management).

2.10 Conclusion of Literature Survey

The literature survey confirms the technological feasibility of piezoelectric kinetic footstep harvesting while highlighting significant systemic shortcomings in existing

implementations. Piezoelectric ceramic transducers provide high energy density under dynamic compressive impact, but require impedance-conscious rectification and supercapacitor buffering. The ESP32 SoC provides the required dual-core compute capability for edge filtering and wireless telemetry, provided that ADC non-linearities and channel conflicts are addressed. Supervised machine learning via Random Forests offers high classification accuracy on multi-dimensional kinetic waveforms, provided that subject-independent validation is strictly maintained. StepCharge AI is specifically architected to synthesize these findings and resolve the identified research gaps through an end-to-end, scientifically defensible platform.

Chapter 3

Problem Definition and Scope

3.1 Problem Statement

In urban public environments and high-foot-traffic facilities, substantial mechanical kinetic energy is continuously imparted onto flooring surfaces by pedestrian movement. In standard architectural infrastructures, this mechanical energy dissipates completely into the flooring substrate as acoustic noise and low-grade thermal vibration. Concurrently, modern smart buildings, transport hubs, and campus facilities require dense deployments of low-power Internet of Things (IoT) sensor nodes, pathway lighting beacons, and foot-traffic monitoring devices. These electronic systems typically rely on disposable chemical batteries, which entail high recurring maintenance costs, finite operational lifetimes, and severe environmental disposal hazards, or require disruptive hardwired AC mains infrastructure.

However, capturing and utilizing ambient human footstep energy presents formidable multidimensional engineering challenges:

1. **Electromechanical Transduction and Storage Mismatch:** Piezoelectric transducers generate high-voltage, low-current, high-impedance AC electrical pulses characterized by transient spikes and low fundamental frequencies (1.0–2.5 Hz). Direct connection to low-impedance storage components or direct-current electronics collapses the terminal voltage, leading to negligible energy transfer. Efficiently rectifying, protecting, and accumulating this irregular energy into an electrostatic storage buffer (such as a supercapacitor) without exceeding voltage ratings or suffering severe leakage represents a significant circuit challenge.

2. **Edge Sensing and Signal Conditioning under Ambient Noise:** Footstep kinetic waveforms are susceptible to high-frequency electromagnetic interference (EMI), mechanical rebound vibrations, and acoustic floor noise. Conventional edge counters that rely on simple threshold comparators routinely suffer from false multiple-triggering or missed detections. Acquiring clean voltage transients on low-power microcontrollers without loading down the high-impedance harvesting rail requires precision signal conditioning and robust digital filtering.

3. **Biomechanical Classification and Data Leakage:** While the temporal amplitude characteristics of footstep pulses correlate with impact intensity and gait

dynamics, existing analytics systems frequently suffer from data leakage. Models trained on random splits memorize individual cadence rhythms rather than generalizable kinetic intensity features.

4. **Real-Time Observability and Verifiable Control:** Most energy harvesting setups operate as blind analog prototypes lacking digital telemetry, real-time cloud data pipelines, or verifiable load actuation. Open-loop control systems falsely assume that sending a network actuation command implies successful physical hardware switching.

Therefore, the core problem is to design, construct, and validate an integrated cyber-physical platform—**StepCharge AI**—that captures kinetic footstep energy, conditions and stores it into a supercapacitor, acquires calibrated dual-channel analog telemetry via an ESP32 microcontroller, securely streams time-series data to a cloud database, classifies footstep impact intensity using machine learning with subject-independent validation, and provides real-time web observability with closed-loop hardware load confirmation.

3.2 Goals and Objectives

The overall goal of StepCharge AI is to deliver a functional, research-grade, and academically defensible IoT kinetic energy harvesting and monitoring platform. The specific operational objectives are:

- **Objective 1 (Physical Transduction):** Assemble a piezoelectric transducer tile capable of converting human heel-strike and toe-off mechanical impacts into electrical charge.
- **Objective 2 (Power Conditioning and Storage):** Implement a low forward-drop Schottky bridge rectifier and a 0.1 F, 5.5 V supercapacitor storage buffer with hardware clamping and software safety guards.
- **Objective 3 (Edge Filtering and Detection):** Program an ESP32 microcontroller to execute 16× ADC oversampling, Exponential Moving Average smoothing ($\alpha = 0.25$), and dual-threshold Schmitt trigger step detection ($V_{\text{high}} = 0.80\,\text{V}$, $V_{\text{low}} = 0.55\,\text{V}$) with a 200–250 ms refractory lockout.
- **Objective 4 (Secure Telemetry Pipeline):** Implement a RESTful Node.js + Express backend that authenticates edge devices using SHA-256 tokens and enforces role-based access control (RBAC).
- **Objective 5 (Cloud Persistence):** Store telemetry frames and footstep events in MongoDB Atlas using compound indices optimized for real-time querying.

- **Objective 6 (Real-Time Push Distribution):** Stream live data to connected web clients via Socket.IO room namespaces.
- **Objective 7 (Machine Learning Classification):** Train a Random Forest model on 5 time-domain physical features to classify footsteps into `LIGHT`, `NORMAL`, and `HEAVY` categories, evaluated using Stratified Group K-Fold cross-validation.
- **Objective 8 (Interactive Web Dashboard):** Build a responsive React 19 + TypeScript dashboard featuring real-time dials, historical charts, AI classification feeds, and closed-loop load state indicators.

3.3 Statement of Scope

The scope of the StepCharge AI preliminary project is precisely bounded as follows:

- **In-Scope:**
  - Design and benchtop fabrication of a single-tile piezoelectric harvester module housing multiple PZT discs.
  - Full-wave Schottky rectification, overvoltage clamping, and supercapacitor storage.
  - Dual-channel analog voltage sensing (piezo pulse voltage on GPIO 34, supercapacitor storage voltage on GPIO 35) via calibrated resistive dividers.
  - ESP32 C++ firmware performing digital filtering, Schmitt trigger step detection, and periodic HTTPS REST telemetry.
  - Node.js, Express, and TypeScript backend architecture with MongoDB Atlas integration and Socket.IO real-time broadcasting.
  - Supervised machine learning training and RESTful inferencing using Random Forests with subject-independent validation.
  - React 19 web application displaying live metrics, analytics, and closed-loop load switching indicators.
  - Demonstration load control of low-power DC devices (LED and 5 V mini DC fan) via N-channel MOSFET drivers.
- **Out-of-Scope (Future Enhancements):**
  - Large-scale building-wide deployment across multiple interconnected hallway tiles.

  - High-power AC mains grid inversion or powering heavy electrical appliances (such as laptops or large industrial motors).
  - On-device edge neural network execution via micro-TensorFlow (inferencing is performed in the Python cloud service).
  - Direct current sensing via physical shunt resistors (power and energy are mathematically estimated from capacitor voltage variations; current is explicitly reported as unavailable).

3.4 Existing System

Existing kinetic energy harvesting installations are predominantly characterized by:

- **Self-Contained Analog Harvesters:** Prototype tiles that connect piezoelectric elements directly to bridge rectifiers and flash an LED or drive an analog voltage meter without digital computation.
- **Non-Instrumented Benchtop Setups:** Academic test rigs driven by electrodynamic vibration shakers producing continuous sinusoidal excitations, which fail to replicate the impulsive, discontinuous nature of human gait.
- **Isolated IoT Sensor Nodes:** Standard IoT monitoring systems that measure ambient building parameters (temperature, humidity) powered entirely by conventional batteries or wall adapters without energy scavenging capability.

3.5 Limitations of Existing Approaches

The limitations inherent in existing systems include:

1. **Zero Longitudinal Observability:** Without digital logging or cloud connectivity, existing systems cannot record cumulative energy yields, degradation trends, or temporal traffic density.
2. **False Triggering in Step Detection:** Fixed analog comparators lack dynamic hysteresis and refractory timing, leading to multiple counts per single footstep due to mechanical rebound.
3. **Methodological Leakage in AI Classification:** Published machine learning gait classifiers commonly employ randomized cross-validation splits, resulting in identity memorization rather than generalizable kinetic intensity detection.

4. **Unverified Load Actuation:** Open-loop IoT controls assume that issuing a toggle request guarantees hardware state transition without confirming physical pin voltages.
5. **Inaccurate Energy Claims:** Conflating peak instantaneous open-circuit voltages with usable deliverable energy without accounting for source impedance, rectifier forward losses, and capacitor ESR.

3.6 Proposed System

The proposed **StepCharge AI** platform resolves these limitations by implementing a unified, multi-tiered architecture that tightly couples physical harvesting with digital intelligence:

- **Physical Layer:** PZT transducer matrix feeding a low forward-drop Schottky bridge rectifier, protected by transient voltage suppression, buffering charge into a 0.1 F / 5.5 V supercapacitor.
- **Embedded Layer:** ESP32 dual-core SoC running firmware that acquires analog voltages at 50 Hz with 16× oversampling, applies Exponential Moving Average smoothing, evaluates footsteps through a dual-threshold Schmitt trigger with hysteresis, and manages closed-loop MOSFET load switching.
- **Cloud & API Layer:** Node.js + Express backend written in TypeScript implementing SHA-256 device authentication, Google OAuth 2.0 user sessions, rate limiting, and MongoDB Atlas time-series persistence.
- **Real-Time Layer:** Socket.IO room namespaces enabling instant telemetry updates to connected clients without polling.
- **AI Analytics Layer:** Python FastAPI microservice providing Random Forest classification trained via subject-independent Stratified Group K-Fold cross-validation across 5 extracted time-domain features.
- **Presentation Layer:** React 19 + TypeScript dashboard rendering live dials, historical charts, AI inference feeds, and verified load states.

3.7 Software Context

StepCharge AI operates at the intersection of renewable energy technology, edge computing, and cloud analytics. Within the smart infrastructure software ecosystem, it functions as an intelligent middleware node that bridges low-level physical

electromechanical transducers with high-level building management systems (BMS), facility occupancy dashboards, and decentralized IoT microgrids.

3.8 Major Constraints

The system operates under several strict physical, embedded, and computational constraints:

1. **Piezoelectric Non-Linearity and Brittleness:** PZT ceramics exhibit non-linear charge generation under high impact forces ($> 800\,\text{N}$) and are prone to mechanical cracking under unbuffered point loads, requiring structural load-distribution plates and elastomeric damping.
2. **Footwear Mechanical Damping:** Footwear compliance introduces significant low-pass mechanical filtering. Thick elastomeric running shoe soles attenuate peak impact voltage by up to 40% and broaden pulse duration compared to hard-soled dress shoes.
3. **ESP32 ADC Non-Linearities:** The built-in 12-bit SAR ADC exhibits non-linear compression below 0.1 V and above 3.1 V under 11 dB attenuation, necessitating precision resistive dividers ($2:1$ ratio) and calibrated voltage mapping.
4. **ADC2 Wi-Fi Conflict:** The ESP32 cannot sample ADC2 channels while the Wi-Fi radio is active. All analog inputs are strictly constrained to ADC1 channels (GPIO 34 and GPIO 35).
5. **Supercapacitor ESR and Leakage:** A 0.1 F supercapacitor possesses non-zero Equivalent Series Resistance (ESR $\approx 25\,\Omega$) and self-discharge leakage (5–15 μA), meaning stored energy dissipates over extended idle periods.
6. **Network Propagation Latency:** Wi-Fi transmissions in dense 2.4 GHz RF environments experience transient retry latencies (50–300 ms), requiring local memory buffering on the ESP32 to prevent telemetry loss.
7. **Measurement Provenance Rule:** When only voltage is sampled across the storage capacitor, the system strictly calculates *estimated stored energy* via $E = \frac{1}{2}CV^2$. Current and instantaneous power are explicitly labeled as `Not Measured` rather than synthesized.

3.9 Methodologies of Problem Solving and Efficiency Issues

To address these technical constraints, the project employs systematic engineering methodologies:

- **Dual-Threshold Schmitt Trigger with Hysteresis:** Prevents noise-induced false counting. A footstep begins only when the rectified voltage exceeds $V_{\text{high}} = 0.80\,\text{V}$ and concludes only after falling below $V_{\text{low}} = 0.55\,\text{V}$.
- **Refractory Lockout Timing:** A 200–250 ms refractory window is enforced following each detected step, ignoring secondary mechanical bounce vibrations.
- **Pulse Duration Boundary Filtering:** Validates that detected pulses satisfy $60\,\text{ms} \leq T_{\text{pulse}} \leq 2000\,\text{ms}$, successfully rejecting electrical noise spikes ($< 60\,\text{ms}$) and static standing weight ($> 2000\,\text{ms}$).
- **Subject-Independent GroupKFold Partitioning:** Groups dataset samples by participant identifier, guaranteeing that no participant's steps appear in both training and testing folds, eliminating gait identity memorization.
- **Closed-Loop GPIO Readback:** The ESP32 drives the MOSFET gate, reads the physical logic level directly from the output pin, and transmits the verified state back to the backend.

3.10 Scenario in Which Multi-Core, Embedded and Distributed Computing Used

StepCharge AI integrates computing paradigms across multiple tiers:

1. **Multi-Core Edge Processing:** The ESP32 incorporates dual 32-bit Xtensa LX6 cores. Core 0 is assigned exclusively to the real-time TCP/IP networking stack, Wi-Fi association, and HTTPS transport. Core 1 runs the time-critical sensor acquisition loop, executing 50 Hz sampling, oversampling accumulation, EMA digital filtering, Schmitt trigger evaluation, and GPIO readback without blocking on network latency.
2. **Embedded Computing:** On-device edge processing filters noise and extracts preliminary temporal properties (pulse peak, average, and duration) directly on the microcontroller, compressing continuous high-frequency analog waveforms into lightweight JSON event packets.
3. **Distributed Cloud Architecture:** The cloud backend is decoupled into independent distributed microservices: a Node.js REST API gateway, a MongoDB

Atlas distributed database replica set, a Python FastAPI machine learning inferencing microservice, and browser-based React clients communicating via WebSockets.

3.11 Expected Outcomes

The expected outcomes of the preliminary engineering phase include:

- A fully assembled, calibrated piezoelectric harvesting tile prototype and power conditioning circuit.
- Operational ESP32 firmware achieving accurate footstep counts and reliable dual-rail voltage sensing.
- A tested Node.js backend supporting secure device authentication and sub-millisecond database queries.
- An operational Socket.IO pipeline pushing real-time events to web clients.
- A trained Random Forest model achieving $> 85\%$ cross-validated accuracy under strict subject-independent validation.
- A responsive web dashboard rendering live dials, historical charts, and verified load switching states.

3.12 Applications

Potential real-world applications of StepCharge AI include:

- **Smart Transit Hubs:** Deploying harvesting tiles in subway turnstiles, airport terminals, and train platforms to power emergency pathway lighting and BLE passenger tracking beacons.
- **Academic and Corporate Campuses:** Installing smart tiles in high-traffic corridors to generate localized auxiliary energy while logging foot-traffic volume.
- **Retail and Commercial Analytics:** Tracking pedestrian density, dwell times, and walking velocity distributions in shopping centers without invasive video cameras.
- **Smart Building Energy Optimization:** Linking foot-traffic cadence to automated HVAC and lighting control systems.

3.13 Hardware Resources Required

The hardware specifications required for the physical prototype and testing environment are detailed in Table 3.1.

Table 3.1: Hardware specifications and prototype bill of materials.

Component	Model / Specifications	Functional Role in StepCharge AI
Microcontroller	Espressif ESP32-WROOM-32D (Dual-Core, 240 MHz, 520 KB SRAM, Wi-Fi/BLE)	Edge signal acquisition, digital filtering, Schmitt trigger detection, and HTTPS telemetry.
Piezoelectric Transducers	35 mm PZT-5 Ceramic Brass Diaphragm Discs (Quantity: 8)	Kinetic compressive impact transduction converting mechanical steps into electrical charge.
Bridge Rectifier	Schottky Diode Bridge (4× 1N5819, $V_F \approx 0.35$ V)	Full-wave AC-to-DC rectification with ultra-low forward voltage loss.
Energy Storage Buffer	0.1 F / 5.5 V Supercapacitor (Electric Double-Layer Capacitor)	Electrostatic storage reservoir buffering harvested kinetic footstep energy.
Overvoltage Protection	5.1 V Zener / TVS Clamping Diode (1N4733A)	Shunts excessive transient voltage spikes to ground, protecting supercapacitor and ADC inputs.
Voltage Dividers	100 kΩ/100 kΩ Metal Film Resistors (2 : 1 ratio, 1% precision)	Attenuates input voltages by half to safely map within the ESP32 0–3.3 V ADC range.
MOSFET Drivers	IRLZ44N / 2N7000 Logic-Level N-Channel MOSFETs	Low-side electronic load switching controlled by ESP32 digital output pins.
Demonstration Loads	High-Efficiency 5 mm Green LED & 5 V Miniature DC Brushless Fan	Visual and mechanical demonstration verifying usable output power delivery.
Development PC	Intel Core i5/i7 (8-Core), 16 GB RAM, 512 GB SSD, Windows 11	C++ firmware compilation, REST API unit testing, and ML model training.

3.14 Software Resources Required

The software stack and development tools utilized across the platform are outlined in Table 3.2.

Table 3.2: Software frameworks, development environments, and toolchain.

Architectural Tier	Technology / Framework	Implementation Purpose / Role
Firmware Toolchain	Arduino IDE / PlatformIO with Espressif ESP32 Core	v2.0.14+; C++ firmware compilation, flash memory partition, and debugging.
Firmware Libraries	`WiFi.h`, `HTTPClient.h`, `ArduinoJson.h`	v7.x; Secure TLS/HTTPS networking, JSON frame serialization, and NTP time sync.
Backend Runtime	Node.js with TypeScript	v20.x LTS; Asynchronous event-driven I/O engine with strict static type safety.
Backend Framework	Express.js	v4.19+; RESTful API routing, rate limiting, and cryptographic middleware.
Database & ODM	MongoDB Atlas with Mongoose ODM	v7.0+; Cloud document store with compound indexing for time-series telemetry.
Real-Time Engine	Socket.IO	v4.7+; Bidirectional WebSocket communication streaming live telemetry updates.
Machine Learning	Python 3.11+, Scikit-Learn, Pandas, NumPy, Joblib	v1.4+; Feature extraction, Stratified GroupKFold cross-validation, Random Forest training.
ML Serving API	FastAPI with Uvicorn ASGI	v0.110+; High-throughput asynchronous HTTP microservice for sub-20ms inference.
Frontend Framework	React 19, TypeScript, Vite	v19.0 / Vite 5; Component hierarchy, reactive store provider, and fast HMR.
Styling & Visuals	Tailwind CSS, Lucide React, Recharts	Responsive dashboard layout, SVG engineering iconography, and live telemetry charts.
Testing Frameworks	Vitest (Backend), Unittest (ML Service)	Automated test harness executing 101 security, boundary, and functional unit tests.

Chapter 4

Software Requirement Specification

4.1 Introduction

4.1.1 Purpose and Scope of the Document

This Software Requirement Specification (SRS) document details the complete functional and non-functional requirements for the **StepCharge AI: Smart Footstep Energy Harvesting and Monitoring System**. Formulated in accordance with the IEEE 830-1998 standard for Software Requirements Specifications, this document defines the operational capabilities, external interfaces, performance targets, database architectures, and design constraints of the system. The scope encompasses the embedded ESP32 C++ firmware, the Node.js + Express + TypeScript REST API backend, the MongoDB Atlas time-series document store, the Python FastAPI machine learning inferencing microservice, and the React 19 + TypeScript client dashboard. It serves as an authoritative technical reference for developers, system architects, and academic evaluators.

4.1.2 Overview of Developer Responsibilities

The multidisciplinary nature of the StepCharge AI platform establishes clear developer responsibilities across four primary engineering domains:

1. **Embedded Systems Developers:** Responsible for writing, testing, and calibrating the ESP32 C++ firmware; configuring 12-bit ADC peripheral sampling with hardware oversampling; implementing Exponential Moving Average (EMA) smoothing; executing dual-threshold Schmitt trigger step detection; maintaining refractory lockout timing; managing non-blocking HTTPS client connections; and reading back physical GPIO pin states for closed-loop load confirmation.

2. **Backend and Cloud Engineers:** Responsible for architecting the asynchronous Node.js REST API; implementing SHA-256 cryptographic device key authentication; configuring Google OAuth 2.0 sessions with secure HTTP-only cookies; enforcing Role-Based Access Control (RBAC); architecting Socket.IO room broadcasting; and designing indexed MongoDB Atlas collections.

3. **Machine Learning Engineers:** Responsible for data curation; designing feature extraction pipelines from kinetic transients; training Random Forest classifiers using subject-independent Stratified Group K-Fold cross-validation; packaging the model into a high-throughput Python FastAPI microservice; and tracking model metadata.
4. **Frontend and UI/UX Engineers:** Responsible for engineering the responsive React 19 dashboard using Tailwind CSS and Recharts; establishing WebSocket state management via Context providers; rendering live telemetry dials and historical trends; implementing keyboard-accessible profile menus; and presenting confirmed hardware load statuses.

4.2 Functional Requirements

4.2.1 Footstep Detection and Hysteresis Thresholding (FR-1)

- **Description:** The edge firmware must continuously detect human footstep impacts from the rectified piezoelectric signal while suppressing electrical noise and mechanical bounce.
- **Input:** Continuous 50 Hz analog voltage readings from GPIO 34 through a 2:1 resistive divider.
- **Processing:** The system accumulates 16 hardware samples per read, computes an Exponential Moving Average ($\alpha = 0.25$), and evaluates a dual-threshold Schmitt trigger. A footstep pulse initiates when the voltage exceeds $V_{\text{high}} = 0.80\,\text{V}$ and concludes when the voltage drops below $V_{\text{low}} = 0.55\,\text{V}$. Pulses outside the duration range of $60\,\text{ms} \leq T_{\text{pulse}} \leq 2000\,\text{ms}$ are rejected. A refractory lockout of 250 ms is enforced after pulse completion.
- **Output:** Discrete footstep event records containing timestamp, peak voltage, average voltage, pulse duration, and cumulative step count.

4.2.2 Dual-Channel Voltage Measurement and Digital Filtering (FR-2)

- **Description:** The edge microcontroller must sample both the instantaneous harvesting pulse rail and the supercapacitor storage buffer rail without channel crosstalk or Wi-Fi radio interference.
- **Input:** Analog voltages on ESP32 ADC1 channels GPIO 34 (Harvest rail) and GPIO 35 (Storage rail).

- **Processing:** Raw 12-bit ADC values (0–4095) are converted to physical voltages using calibrated reference parameters ($V_{\text{ref}} = 3.30\,\text{V}$, divider ratio $M = 2.0$). Moving average smoothing attenuates high-frequency RF noise.
- **Output:** Calibrated floating-point voltages for both rails, refreshed at 1-second telemetry intervals.

4.2.3 Energy Storage Monitoring and Safety Thresholds (FR-3)

- **Description:** The system must compute the electrostatic energy stored in the 0.1 F supercapacitor and enforce safety threshold alerts.
- **Input:** Real-time storage rail voltage (V_{storage}).
- **Processing:** Calculates stored energy in Joules using $E = \frac{1}{2}CV_{\text{storage}}^2$, where $C = 0.10\,\text{F}$. Compares V_{storage} against hard limits: normal ($2.0\,\text{V} \leq V \leq 4.7\,\text{V}$), warning ($V > 4.7\,\text{V}$), and critical ceiling ($V \geq 5.0\,\text{V}$).
- **Output:** Stored energy estimate (E), voltage status category, and overvoltage warning notifications.

4.2.4 Real-Time Machine Learning Inference and Classification (FR-4)

- **Description:** The system must classify each valid footstep impact into one of three intensity categories: `LIGHT`, `NORMAL`, or `HEAVY`.
- **Input:** Kinetic feature vector: $[V_{\text{peak}}, V_{\text{avg}}, T_{\text{pulse}}, \Delta T_{\text{step}}, V_{\text{storage}}]$.
- **Processing:** Dispatched to the Python FastAPI microservice, which executes inference via a pre-trained Random Forest model and outputs class probabilities.
- **Output:** Predicted class label and prediction confidence score (0.0–1.0). If the ML microservice is unreachable, the system gracefully records `null` for prediction without failing telemetry persistence.

4.2.5 ESP32 Telemetry Transmission and Reconnection Buffer (FR-5)

- **Description:** Edge devices must package telemetry and transmit it over HTTPS to the backend REST API every 1000 ms.
- **Input:** Latest telemetry state and pending footstep event queues.
- **Processing:** Serializes payloads into JSON format, attaches the pre-shared device authentication key via the `X-Device-Key` header, and executes an HTTPS

POST request. If transmission fails, footstep events are retained in a circular buffer in SRAM and resent upon reconnection.

- **Output:** HTTP 200 OK response from the backend; updated device uptime and link quality.

4.2.6 User Authentication and Role-Based Access Control (FR-6)

- **Description:** Secure authentication for web dashboard users, supporting Google OAuth 2.0 and local credentials with RBAC enforcement.
- **Input:** Google OAuth authorization code or email/password credentials.
- **Processing:** Verifies credentials, retrieves or provisions the user in MongoDB Atlas, issues a signed JWT token containing user identity and role (`USER` or `ADMIN`), and sets an HTTP-only, secure, SameSite session cookie.
- **Output:** Authenticated session profile and authorization headers for protected routes.

4.2.7 Real-Time Dashboard Observability (FR-7)

- **Description:** Web clients must display live hardware telemetry and AI predictions without requiring manual browser refreshing.
- **Input:** Socket.IO WebSocket event stream on room `device:<deviceId>`.
- **Processing:** React context store consumes incoming `telemetry:update` and `footstep:detected` events and updates graphical widgets.
- **Output:** Dynamic animated dials (instantaneous voltage, supercapacitor level), real-time footstep pulse stream, and recent AI classification badges.

4.2.8 Historical Telemetry Logging and Export (FR-8)

- **Description:** Users must be able to query, filter, and export historical energy harvesting and footstep records.
- **Input:** Query parameters (date range, device identifier, step class).
- **Processing:** Backend executes indexed queries on MongoDB Atlas, aggregates temporal metrics, and formats data into JSON or downloadable CSV files.
- **Output:** Tabular data views and downloadable CSV datasets for research analysis.

4.2.9 Closed-Loop Load Actuation and Feedback Verification (FR-9)

- **Description:** Authorized users can actuate demonstration loads (LED indicator, 5 V DC fan), with the UI visually reflecting state changes only upon verified physical GPIO readback.
- **Input:** User toggle action on dashboard; load key (`led` or `fan`) and requested state (`true`/`false`).
- **Processing:** Backend persists command intent. The ESP32 polls the command state, switches the corresponding MOSFET gate, reads the digital logic level directly back from the pin, and reports `actualState` to the backend.
- **Output:** The dashboard renders the load as active only when `actualState == true`, eliminating open-loop actuation discrepancies.

4.2.10 Automated System Health and Anomaly Alerts (FR-10)

- **Description:** The backend must monitor device health, detect anomalous sensor readings, and generate actionable alerts.
- **Input:** Ingested telemetry streams and periodic health watchdog checks.
- **Processing:** Identifies sensor flat-lining ($V < 0.02\,\text{V}$ for > 5 minutes under expected traffic), storage overvoltage ($V > 4.70\,\text{V}$), or device offline status ($>$ 30 seconds without telemetry). Persists alerts in MongoDB and emits `alert:created` via Socket.IO.
- **Output:** Real-time alert notifications on the dashboard with "Mark as Read" acknowledgment capabilities.

4.3 Non-Functional Requirements

4.3.1 Performance Requirements

- **Edge Sampling Rate:** The ESP32 analog sampling loop must execute at a deterministic frequency of 50 Hz (20 ms $\pm$ 1 ms interval).
- **Telemetry Latency:** Edge-to-cloud telemetry transmission must complete within 500 ms under standard Wi-Fi conditions.
- **Real-Time Push Latency:** The Socket.IO server must broadcast incoming telemetry frames to connected browser clients within 100 ms of ingestion.

- **ML Inference Throughput:** The Python FastAPI microservice must process single-sample inference requests within 25 ms.
- **UI Responsiveness:** The dashboard must maintain 60 frames per second (FPS) rendering on modern browsers during live high-frequency data streaming.

4.3.2 Reliability and Fault Tolerance

- **Graceful Degradation:** If the Python ML microservice becomes unavailable, the backend must continue processing and persisting telemetry, recording footsteps with `null` classifications without throwing fatal 500 errors.
- **Offline Edge Buffering:** If Wi-Fi connectivity drops, the ESP32 must continue step counting and buffer up to 100 recent footstep events in SRAM, flushing them upon link restoration.
- **Watchdog Protection:** The ESP32 firmware must utilize a hardware watchdog timer (8-second timeout) to automatically reboot in the event of an unrecoverable freeze.

4.3.3 Security Requirements

- **Cryptographic Transport:** All communication between edge devices, backend servers, and client browsers must be strictly encrypted via TLS 1.3 / HTTPS and WSS (secure WebSockets).
- **Device Key Hashing:** Edge device authentication keys are never stored in plaintext. The backend hashes incoming `X-Device-Key` values using SHA-256 before performing constant-time comparisons against database records.
- **Session Cookie Protection:** User authentication cookies must be set with `HttpOnly`, `Secure` (in production), and `SameSite=Lax` (or `None` for cross-origin HTTPS) attributes to mitigate Cross-Site Scripting (XSS) and Cross-Site Request Forgery (CSRF).
- **Rate Limiting:** Public and sensitive endpoints must be protected against brute-force attacks via Express rate limiters (120 requests/minute for telemetry; 10 requests/15 minutes for authentication).
- **Audit Logging:** Administrative operations, user logins, load actuation requests, and device reconfigurations must be immutably recorded in an `audit_logs` collection.

4.3.4 Availability

The cloud backend services and MongoDB Atlas database must be architected for high availability, targeting a minimum operational uptime of 99.5% during continuous experimental observation periods.

4.3.5 Scalability

The database schema must utilize compound indexes on `{ deviceId: 1, timestamp: -1 }` to ensure that query execution time remains $O(\log N)$ as the telemetry collection scales to millions of documents. Socket.IO communication must be structured into device-specific room channels, preventing broadcast storms.

4.3.6 Usability

The client dashboard must adhere to modern accessibility and responsive design standards (WCAG 2.1 Level AA compliance), featuring high-contrast dark theme surfaces, legible typography, keyboard-navigable menus (Escape key, outside click detection), and intuitive visual feedback states (default, hover, active, loading, error).

4.3.7 Maintainability

The codebase must enforce strict modular decoupling: TypeScript interfaces across backend controllers, PEP-8 compliance in Python microservices, and reusable component hierarchies in React, supported by automated linting (oxlint/eslint) and Vitest test suites.

4.4 Hardware Requirements

The hardware requirements for deploying the complete StepCharge AI system are summarized as follows:

- **Harvester Module:** 8× 35 mm PZT-5 piezoelectric ceramic diaphragm discs wired in a series-parallel array, integrated into a load-distributing mechanical flooring tile.
- **Power Conditioning Circuit:** 4× 1N5819 Schottky diodes, 5.1 V Zener overvoltage clamp, 0.1 F / 5.5 V supercapacitor, and dual 2:1 precision resistive dividers (100 kΩ/100 kΩ, 1% tolerance).

- **Processing Board:** Espressif ESP32-WROOM-32D development board powered via 5 V micro-USB.
- **Actuation Circuit:** 2× IRLZ44N N-channel logic-level MOSFETs driving a 5 mm green LED and a 5 V miniature DC brushless demonstration fan.
- **Networking Infrastructure:** 2.4 GHz Wi-Fi access point supporting IEEE 802.11 b/g/n with WPA2-PSK security.

4.5 Software Requirements

The operational software environments include:

- **Operating System:** Cross-platform compatibility; developed and tested on Windows 11 64-bit and Ubuntu 22.04 LTS.
- **Runtimes and Compilers:** Node.js v20.x LTS, Python v3.11+, and GNU C++ (Xtensa toolchain via ESP-IDF / Arduino core).
- **Cloud Services:** MongoDB Atlas Serverless/M2 Cluster running MongoDB 7.0+.
- **Client Browsers:** Modern Chromium-based browsers (Google Chrome 120+, Microsoft Edge) or Mozilla Firefox 120+ with WebSocket support.

4.6 Database Requirements

The database layer requires a flexible, high-performance document store capable of ingesting high-frequency time-series data while managing relational associations between users and devices:

- **Database Engine:** MongoDB Atlas 7.0+.
- **Object Data Modeling (ODM):** Mongoose ODM v8.x for schema validation, type safety, and lifecycle middleware.
- **Primary Collections:**
    1. `users`: Manages user credentials, Google OAuth profile identifiers, roles (`USER`, `ADMIN`), and session timestamps.
    2. `devices`: Stores edge device identities, hashed API keys (`apiKeyHash`), ownership mappings, load states, and online statuses.
    3. `telemetry`: High-frequency time-series collection storing periodic rail voltages, stored energy, and link metrics.

4. `footsteps`: Event-driven collection storing discrete step records, physical features, AI classifications, and confidence scores.
5. `alerts`: Records system warnings, overvoltage thresholds, and sensor faults with read/unread statuses.
6. `dataset_samples`: Curated ground-truth footstep feature vectors labeled for model retraining.
7. `model_metadata`: Registry of trained machine learning model artifacts, hyperparameters, and evaluation metrics.
8. `audit_logs`: Tamper-evident operational audit trail recording security events and load toggles.

- **Indexing Strategy:** Compound indexes on `{ deviceId: 1, timestamp: -1 }` across `telemetry` and `footsteps` to optimize range queries and chronological charting.

4.7 System Constraints

The system implementation is constrained by the following architectural boundaries:

1. **Direct Energy Measurement Boundary:** The system directly measures terminal voltages across calibrated resistive dividers. Stored energy is calculated via $E = \frac{1}{2}CV^2$. Current and instantaneous power are not measured and are explicitly omitted from telemetry contracts to preserve scientific integrity.
2. **ADC Attenuation Limit:** The ESP32 ADC1 is configured with 11 dB attenuation, providing a measurable range of 0.0–3.1 V. The external 2 : 1 divider scales external voltages up to 6.0 V into the safe measurable window. Voltages exceeding 5.5 V are clamped by the physical Zener diode.
3. **Single-Tile Physical Testing:** Preliminary prototype evaluation is conducted on a single instrumented tile module rather than an extensive multi-tile network.

4.8 Development Methodology: The Agile SDLC Model

4.8.1 Justification of the Agile Methodology

The development of StepCharge AI adopted the Agile Software Development Life Cycle (SDLC) model. Developing an advanced cyber-physical system that bridges

analog electromechanical harvesting, embedded C++ firmware, cloud web services, and machine learning involves high technical uncertainty and inter-component dependencies. Linear sequential models (such as the traditional Waterfall model) are ill-suited for this project because physical hardware discoveries (such as unexpected sensor noise, ADC non-linearities, or footwear damping) directly necessitate iterative firmware and backend adjustments.

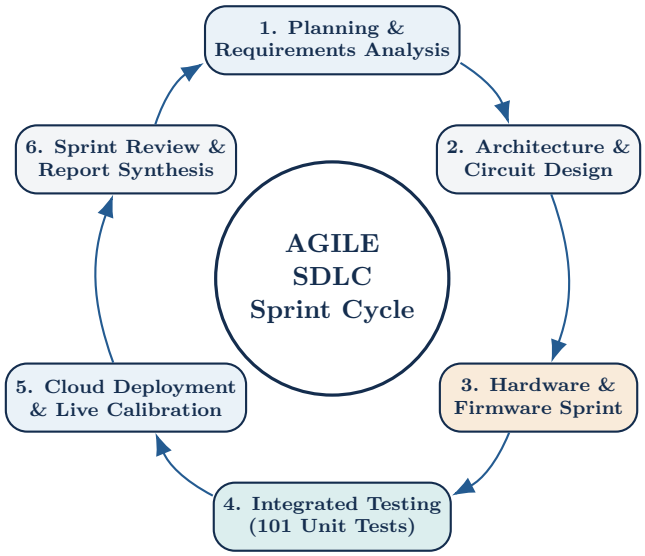


Figure 4.1: The iterative Agile Software Development Life Cycle (SDLC) model applied to StepCharge AI.

The Agile approach provided three critical operational benefits:

1. **Incremental Functional Deliverables:** Each sprint produced a verified subsystem milestone. Sprint 1 established the physical rectification and benchtop voltage logging; Sprint 2 established the ESP32 Schmitt trigger and local filtering; Sprint 3 delivered the Node.js API and MongoDB database; Sprint 4 integrated Socket.IO and the React dashboard; and Sprint 5 integrated the Python machine learning service.

2. **Rapid Experimental Feedback Loops:** Empirical observations from physical walking tests were incorporated into subsequent software iterations. For instance, initial tests revealed electrical noise spikes falsely incrementing step counts, prompting the introduction of the 60 ms minimum pulse width filter and 250 ms refractory lockout in the subsequent sprint.

3. **Continuous Test-Driven Integration:** Automated testing (Vitest backend suite, ML unit tests) was maintained continuously across sprints, ensuring that architectural enhancements (such as migrating authentication and hardening security) never introduced regressions into existing telemetry pipelines.

4.8.2 Implementation Phases in Agile

The development trajectory was structured into six distinct phases:

- **Phase 1: Planning and Requirements Analysis:** Defining project scope, electromechanical calculations, IEEE SRS formulation, and establishing component specifications.
- **Phase 2: Architectural and Circuit Design:** Schematic drafting, PCB/breadboard layout, database schema definition, and API endpoint specification.
- **Phase 3: Development Sprints:** Parallel development of ESP32 C++ firmware, Node.js REST controllers, MongoDB Atlas integration, React frontend components, and Python ML pipelines.
- **Phase 4: Integrated Testing and Calibration:** Benchtop sensor characterization, dual-rail divider calibration, security penetration audits, and 101 automated Vitest unit tests.
- **Phase 5: Cloud Deployment and Real-Time Verification:** Deploying backend services, establishing Socket.IO namespaces, verifying Google OAuth sessions, and live hardware-in-the-loop stress testing.
- **Phase 6: Sprint Review and Academic Evaluation:** Feature validation against SRS criteria, performance assessment, and preliminary project report synthesis.

Chapter 5

System Design

5.1 System Architecture

The system architecture of **StepCharge AI** follows an end-to-end cyber-physical design structured into four decoupled operational tiers: Physical Harvesting & Edge Hardware, Embedded Firmware & Sensing, Cloud Backend & Data Persistence, and Machine Learning Analytics & Presentation Clients. Figure 5.1 illustrates the architectural topology and multi-protocol dataflow connecting physical footstep impacts to real-time client visualization.

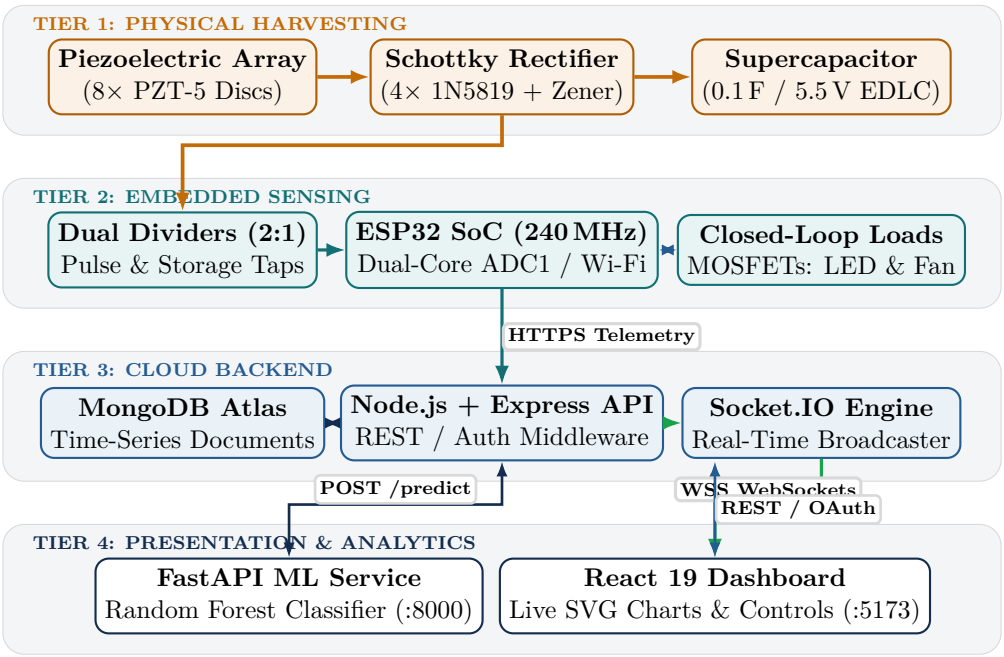


Figure 5.1: Overall StepCharge AI multi-tier cyber-physical system architecture.

The architecture ensures that analog sensing and power conditioning remain completely decoupled from high-level cloud services. Even during transient network outages, local energy accumulation and step detection continue uninterrupted on the edge hardware.

5.2 Overall Hardware Architecture

The hardware subsystem integrates kinetic energy harvesting, analog conditioning, digital processing, and power switching. Figure 5.2 provides a structural block diagram of the physical components.

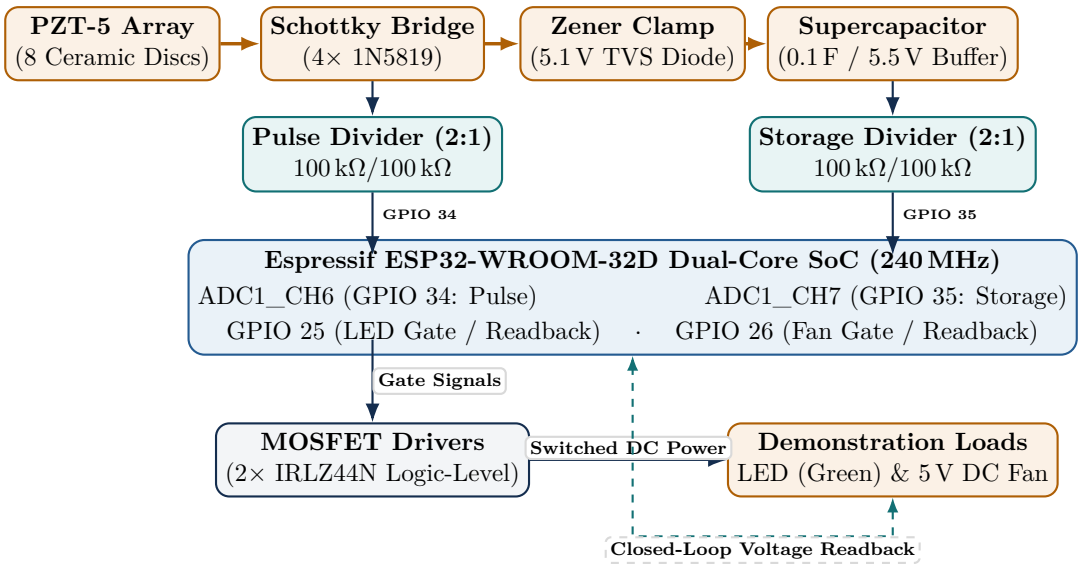


Figure 5.2: Detailed hardware block diagram of the StepCharge AI edge harvesting node.

5.3 Piezoelectric Energy Harvesting Subsystem

The harvesting array consists of 8 commercial 35 mm circular Lead Zirconate Titanate (PZT-5) diaphragm elements mounted onto flexible brass backing plates. The transducers are wired in a series-parallel matrix (2×4) embedded within an elastomeric polyurethane frame beneath a rigid 300 mm × 300 mm acrylic top plate. When a footstep impact lands on the plate, mechanical force is distributed across the ceramic elements, inducing dynamic flexural bending and generating simultaneous charge outputs.

5.4 Rectifier and Power Conditioning Circuit

Because piezoelectric transducers generate bipolar alternating current (AC), rectification is mandatory. Figure 5.3 depicts the electrical circuit schematic of the conditioning and measurement stages.

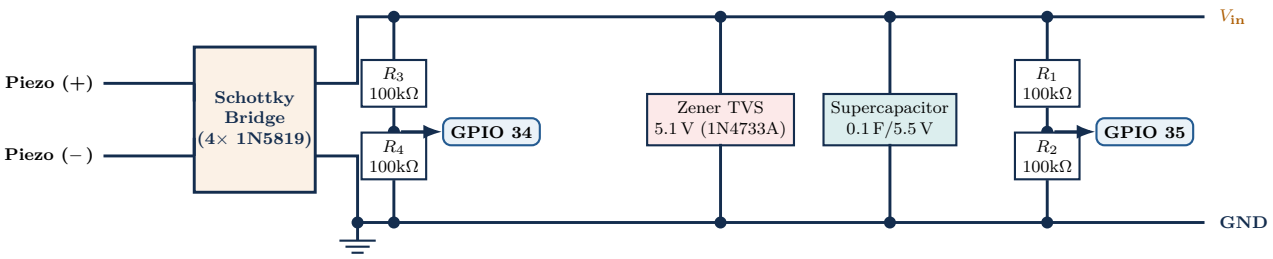


Figure 5.3: Electrical schematic of rectification, clamping, supercapacitor, and dual-divider sensing.

The Schottky diodes (1N5819) exhibit an ultra-low forward-voltage drop ($V_F \approx 0.35\,\text{V}$), conserving scarce micro-energy during low-velocity footfalls. A 5.1 V Zener clamping diode shunts excessive voltage transients to ground, preventing damage to the supercapacitor buffer and the microcontroller ADC.

5.5 Supercapacitor Energy Storage

Energy accumulation utilizes a 0.1 F, 5.5 V Electric Double-Layer Capacitor (EDLC). The total electrical energy stored electrostatically in the capacitor dielectric is given by:

$$E = \frac{1}{2}CV^2 \tag{5.1}$$

where $C = 0.10\,\text{F}$ and V is the terminal voltage. The differential energy accumulated between consecutive footstep cycles is:

$$\Delta E = \frac{1}{2}C\left(V_{\text{final}}^2 - V_{\text{initial}}^2\right) \tag{5.2}$$

5.6 Voltage Measurement Subsystem

To interface external potentials with the ESP32 (0.0–3.3 V absolute maximum), symmetric 100 kΩ/100 kΩ metal-film resistive dividers are implemented, establishing a scaling ratio of:

$$M = \frac{R_1 + R_2}{R_2} = \frac{100\,\text{k}\Omega + 100\,\text{k}\Omega}{100\,\text{k}\Omega} = 2.0 \tag{5.3}$$

The true physical voltage V_{phys} is calculated from the raw 12-bit ADC integer $D \in [0, 4095]$ as:

$$V_{\text{phys}} = \left(\frac{D}{4095.0}\right) \times 3.30\,\text{V} \times 2.0 \tag{5.4}$$

5.7 ESP32 Embedded Monitoring and Signal Acquisition

The firmware executes a dual-threshold Schmitt trigger to detect valid footsteps. Figure 5.4 illustrates the hysteresis detection mechanics.

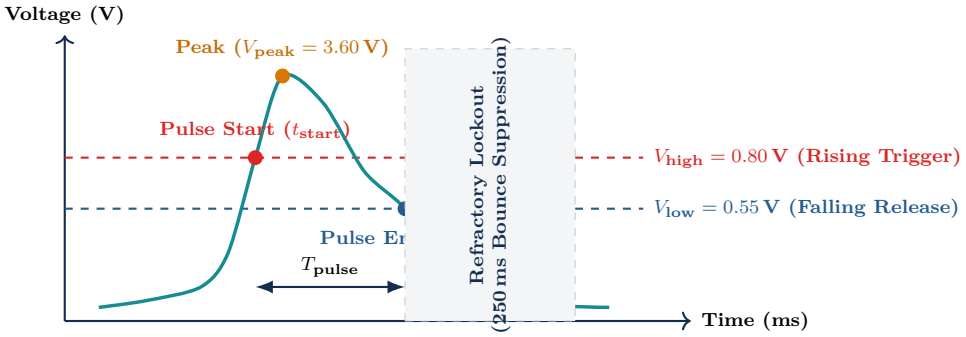


Figure 5.4: Schmitt-trigger hysteresis detection waveform and refractory lockout window.

5.8 IoT Communication Architecture

The edge node communicates with the cloud backend via standard TLS-encrypted HTTPS REST protocols. Edge telemetry frames are published every 1000 ms via `POST /api/devices/:id/telemetry`. Discrete footsteps are appended via `POST /api/devices/:id/foot`

5.9 Backend Architecture

The backend service is structured into modular layers:

- **Routing Layer:** Express routers handling the REST routes for `/api/auth`, `/api/devices`, `/api/telemetry`, `/api/alerts`, and `/api/ml`.
- **Middleware Layer:** Cryptographic device key validation (`requireDeviceAuth`), JWT session verification (`requireAuth`), RBAC authorization (`requireAdmin`), rate limiters, and Helmet security headers.
- **Service Layer:** Telemetry persistence, health monitoring, and Socket.IO emission.

5.10 Database Design

MongoDB Atlas provides document persistence. Relational integrity is enforced programmatically via Mongoose ODM schemas. The primary entity relationships are shown in Section 5.14.

5.11 Machine Learning Architecture

The machine learning architecture runs as an independent Python FastAPI microservice. The model takes a 5-element feature vector and classifies it via a Random Forest model. Figure 5.5 illustrates both the offline training pipeline and the online real-time inference workflow.

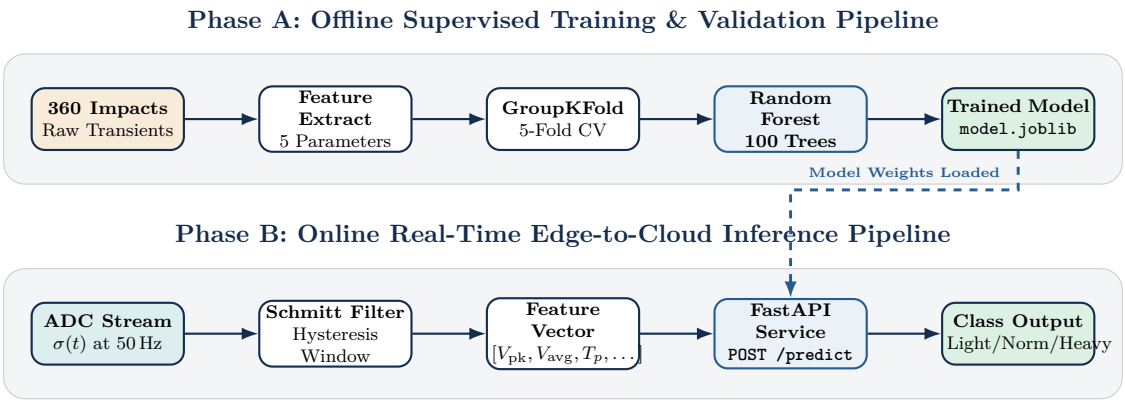


Figure 5.5: Decoupled Machine Learning architecture: offline validation and online real-time inference.

5.12 Frontend Dashboard Architecture

The frontend is constructed using React 19 and TypeScript, structured around a global `StoreProvider`. A custom hook (`useStore`) connects to the Socket.IO client, dynamically receiving telemetry frames and updating charts without full re-renders.

5.13 Data Flow Diagrams

5.13.1 Data Flow Diagram Level 0 (Context Level)

Figure 5.6 illustrates the Level 0 Context DFD representing the highest-level interaction between external entities and the StepCharge AI platform.

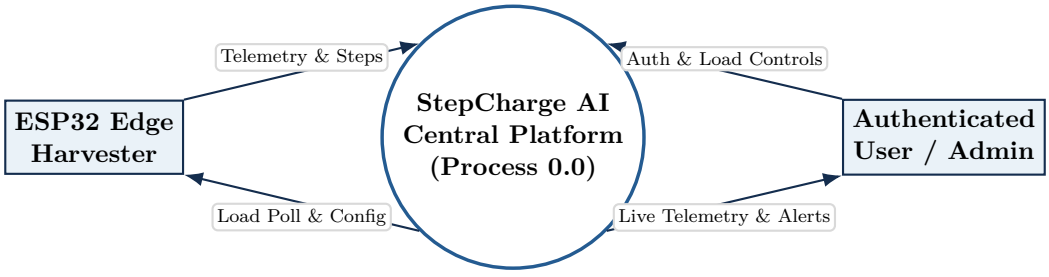


Figure 5.6: Data Flow Diagram (DFD) Level 0 — Context Level Diagram.

5.13.2 Data Flow Diagram Level 1

Figure 5.7 decomposes the system into discrete functional processes, illustrating data stores and internal data paths.

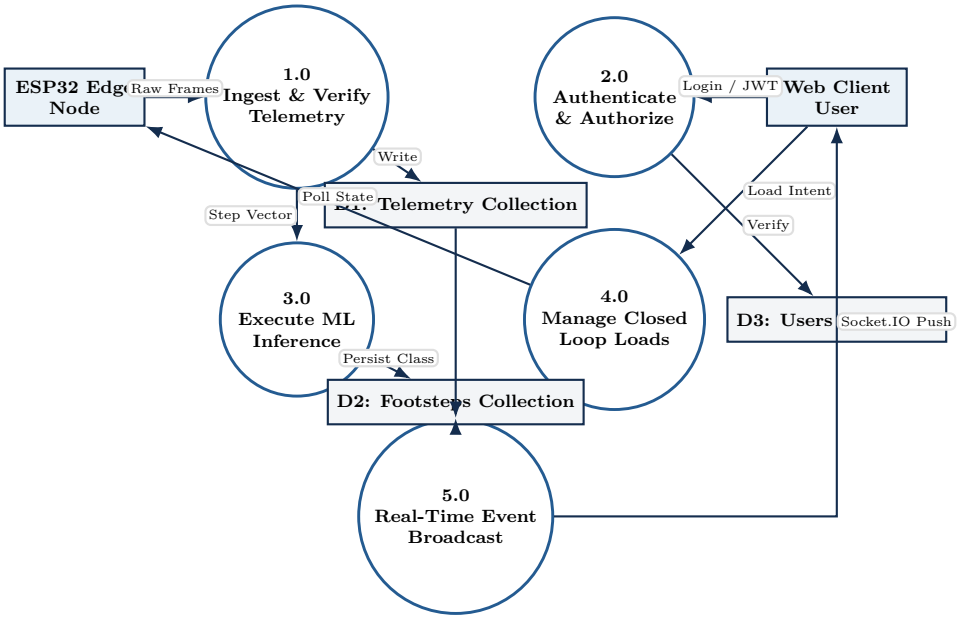


Figure 5.7: Data Flow Diagram (DFD) Level 1 — Functional Process Decomposition.

5.14 Entity Relationship Diagram

Figure 5.8 presents the Entity-Relationship (ER) model defining the primary collections, attributes, and cardinality in MongoDB Atlas.

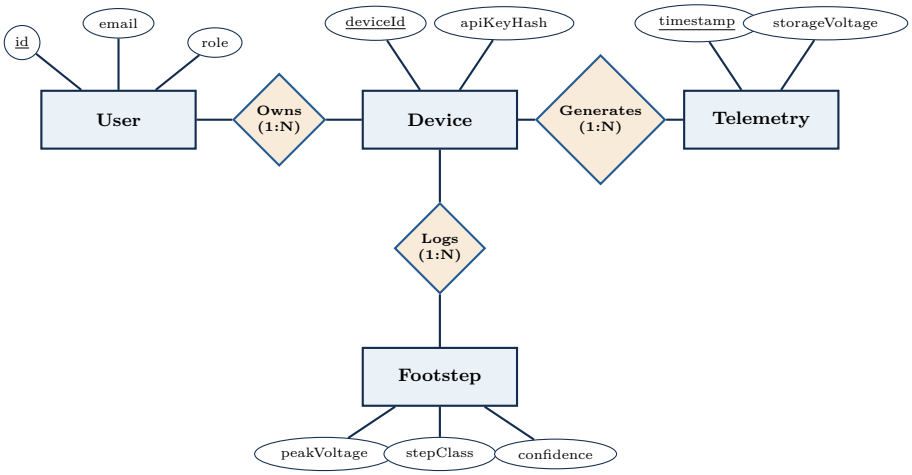


Figure 5.8: Entity-Relationship (ER) diagram representing the MongoDB Atlas schema.

5.15 Unified Modeling Language (UML) Diagrams

5.15.1 Use Case Diagram

Figure 5.9 depicts system interactions from the perspective of two primary actors: the Pedestrian (who imparts mechanical stress) and the Authenticated System User / Admin (who monitors and manages the system).

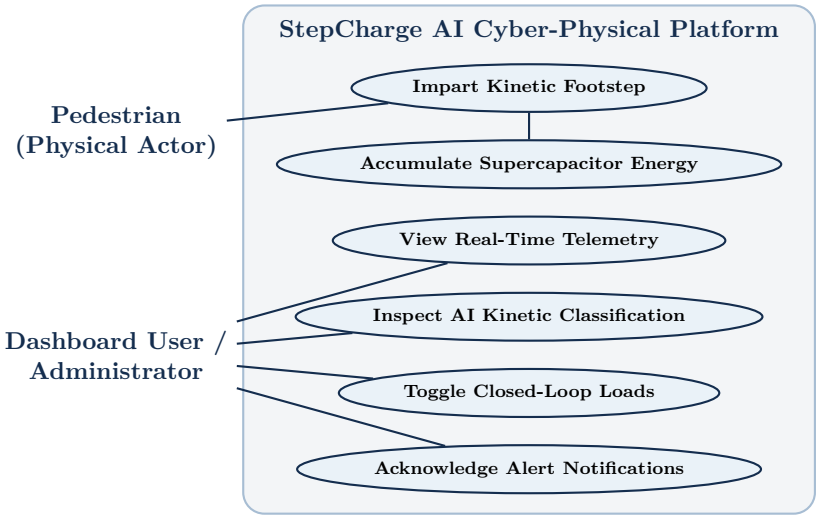


Figure 5.9: Use Case Diagram modeling user and physical actor interactions.

5.15.2 Activity Diagram

Figure 5.10 traces the continuous operational lifecycle of edge signal acquisition, Schmitt trigger validation, and telemetry streaming.

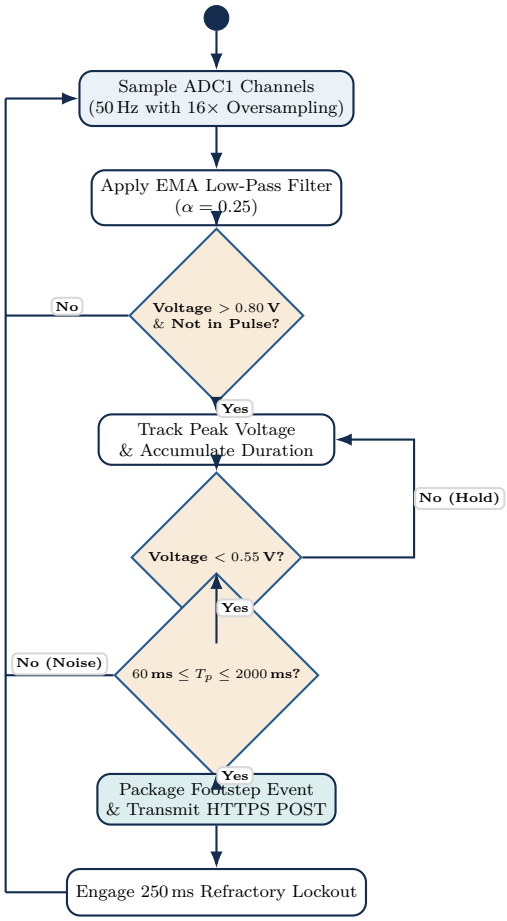


Figure 5.10: Activity Diagram illustrating the edge firmware step detection and telemetry cycle.

5.15.3 State Machine Diagram

Figure 5.11 represents the operational states of the ESP32 edge microcontroller.

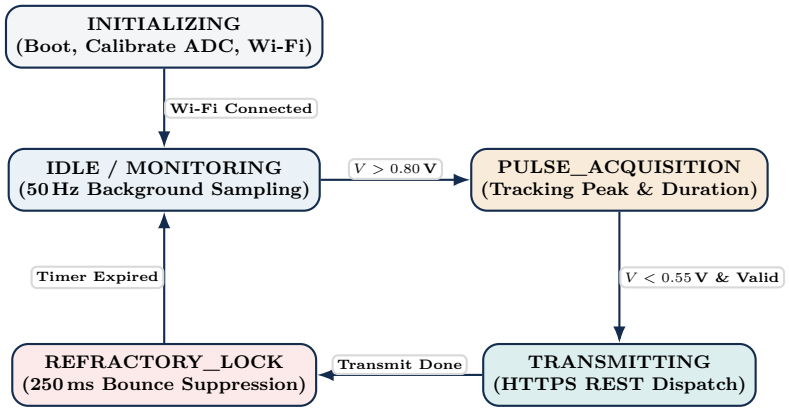


Figure 5.11: State Machine Diagram of the ESP32 edge firmware.

5.15.4 Class Diagram

Figure 5.12 outlines the object-oriented structure of backend models and controllers.

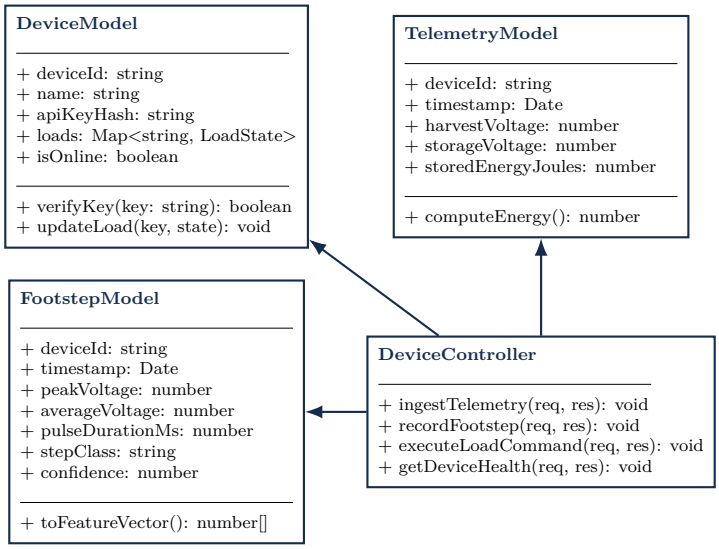


Figure 5.12: Class Diagram illustrating primary backend models and services.

5.15.5 Sequence Diagram

Figure 5.13 models the interaction sequence during footstep detection, ML inferencing, and live broadcasting.

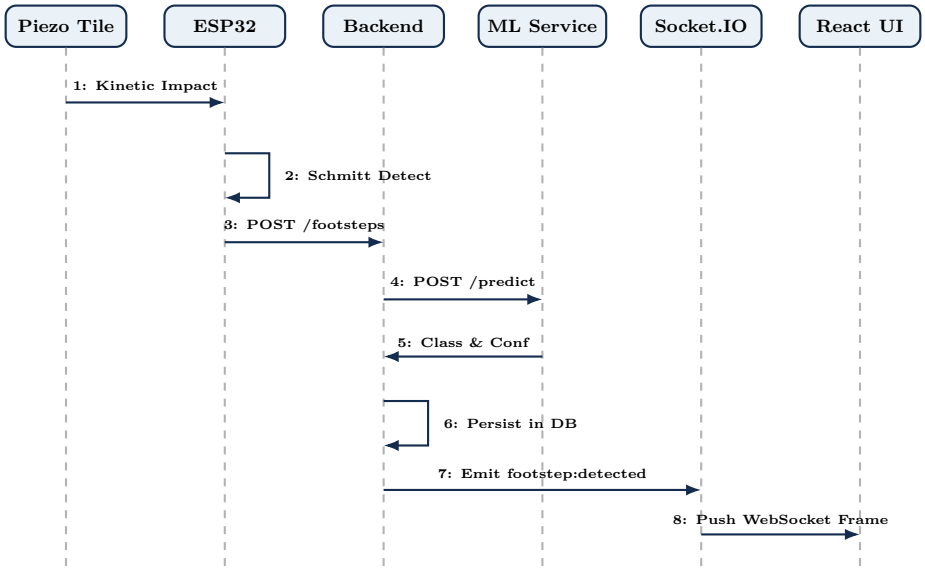


Figure 5.13: Sequence Diagram of footstep acquisition, ML classification, and UI update.

5.15.6 Object Diagram

Figure 5.14 depicts a concrete runtime snapshot of system instances during a live testing session.

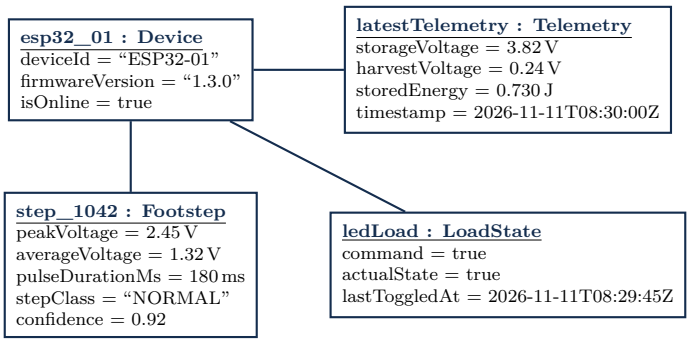


Figure 5.14: Object Diagram capturing runtime instances during a footstep event.

Chapter 6

Implementation and Technical Specifications

6.1 Introduction

The implementation of StepCharge AI translates the theoretical design into a verified, operational cyber-physical system. This chapter details the physical prototyping of the piezoelectric floor tile, embedded C++ firmware routines, cloud REST controllers in TypeScript, MongoDB Atlas schema integration, the Python machine learning inferencing microservice, and the React 19 web dashboard. Furthermore, the chapter documents the security architectures, hardware-software integration loops, and error-handling protocols established across the platform.

6.2 Hardware Implementation and Prototyping

The physical prototype was constructed using an iterative benchtop prototyping methodology. The mechanical structure consists of a sandwich enclosure:

1. **Base Substrate:** A 12 mm high-density plywood baseplate providing structural rigidity and dampening acoustic reflections from the floor.
2. **Transducer Matrix Layer:** Eight 35 mm Lead Zirconate Titanate (PZT-5) piezoelectric diaphragm discs arranged in a 2×4 rectangular grid. Each disc is seated on high-resilience silicone O-rings to permit controlled flexural deformation without edge pinning.
3. **Impact Transmission Plate:** A 6 mm transparent polymethyl methacrylate (acrylic) surface plate that distributes the pedestrian's vertical heel-strike load uniformly across the underlying transducers.
4. **Perimeter Shock Absorbers:** Neoprene elastomeric foam perimeter gaskets that absorb lateral shear forces and restore the acrylic plate to its neutral equilibrium state following impact release.

6.3 Piezoelectric Sensor Integration

The eight piezoelectric discs are electrically interconnected in a series-parallel configuration. Pairing two discs in series doubles the open-circuit output voltage,

while connecting four parallel branches quadruples the aggregate output current. The combined positive and negative terminals are shielded with braided copper sleeving to minimize 50 Hz electromagnetic interference (EMI) pick-up before routing into the signal conditioning circuit.

6.4 Rectification, Conditioning and Supercapacitor Storage

The conditioning circuit is assembled on a dedicated prototyping printed circuit board (PCB):

- **Rectification:** A full-wave bridge rectifier constructed with four 1N5819 Schottky barrier diodes. The low forward-voltage drop ($V_F \approx 0.35\,\text{V}$) ensures that low-energy footfalls are efficiently rectified.
- **Voltage Clamping:** A 5.1 V Zener diode (1N4733A) is connected across the rectified DC rail in reverse breakdown configuration, safely clamping voltage spikes exceeding 5.1 V to protect the supercapacitor and the ESP32.
- **Energy Storage Reservoir:** An electric double-layer supercapacitor rated at 0.1 F and 5.5 V is positioned downstream of the rectifier.
- **Sensing Dividers:** Two separate precision resistive dividers (100 kΩ/100 kΩ, metal-film, 1% tolerance) scale the rectified pulse rail and the supercapacitor terminal voltage by a factor of 2.0.

6.5 ESP32 Firmware Implementation

The edge firmware is implemented in C++ using the Arduino-ESP32 core (v2.0.14+). The firmware leverages the ESP32 dual-core Xtensa architecture:

- **Core 0 (Networking Task):** Runs the underlying FreeRTOS TCP/IP stack, Wi-Fi reconnection state machine, and periodic HTTPS REST client requests.
- **Core 1 (Real-Time Sensing Task):** Dedicated to time-critical analog signal sampling, digital filtering, Schmitt-trigger hysteresis tracking, and closed-loop GPIO load control.

6.6 Sensor Data Acquisition and Calibration

Analog sampling executes at a deterministic 50 Hz interval (20 ms) via high-resolution hardware timers. The sampling pipeline incorporates:

1. **16× Hardware Oversampling:** Sixteen consecutive analog reads are performed in rapid succession on ADC1_CH6 (GPIO 34) and ADC1_CH7 (GPIO 35) and averaged to eliminate random Gaussian quantization noise.

2. **Exponential Moving Average (EMA) Smoothing:** The averaged reading is smoothed using the recurrence relation:

$$V_{\text{ema}}[k] = V_{\text{ema}}[k-1] + \frac{V_{\text{raw}}[k] - V_{\text{ema}}[k-1]}{4} \tag{6.1}$$

   which implements a low-pass digital filter with $\alpha = 0.25$, attenuating high-frequency electrical switching spikes.

3. **Calibrated Voltage Conversion:** Calibrated reference coefficients scale raw integer values (0–4095) into physical voltages ($V_{\text{phys}} = \frac{D}{4095.0} \times 3.30\,\text{V} \times 2.0$).

6.7 Wi-Fi Communication and Secure REST Telemetry

The ESP32 connects to an encrypted 2.4 GHz WPA2 Wi-Fi network. Every 1000 ms, the firmware constructs a structured JSON telemetry frame and dispatches it via an HTTPS POST request:

Listing 6.1: JSON telemetry serialization in ESP32 firmware.

```
JsonDocument doc;
doc["storageVoltage"] = lastStorage;
doc["harvestVoltage"] = lastPeak;
doc["storedEnergyJoules"] = 0.5 * 0.10 * lastStorage * lastStorage;
doc["footstepCount"]  = footstepCount;
doc["uptimeSec"]      = millis() / 1000;

String jsonPayload;
serializeJson(doc, jsonPayload);

http.begin(client, SERVER_TELEMETRY_URL);
http.addHeader("Content-Type", "application/json");
http.addHeader("X-Device-Key", DEVICE_SECRET_KEY);
int httpResponseCode = http.POST(jsonPayload);
```

6.8 Backend API Implementation

The backend service is engineered in Node.js, Express, and TypeScript. The application architecture enforces modular separation of concerns:

- `/backend/src/controllers/deviceController.ts`: Ingests incoming telemetry, validates physical voltage bounds ($0.0\,\text{V} \leq V \leq 60.0\,\text{V}$), ensures $V_{\text{avg}} \leq V_{\text{peak}}$, and commits records to MongoDB Atlas.

- `/backend/src/controllers/authController.ts`: Manages Google OAuth 2.0 authorization, credential verification, and user session lifecycle.
- `/backend/src/controllers/alertController.ts`: Generates real-time anomaly alerts and manages the "Mark as Read" operational flow with complete database persistence.

6.9 Google Authentication and Role-Based Access Control

User authentication utilizes Google OAuth 2.0 as the primary identity provider, supplemented by local cryptographic email/password authentication:

1. The client initiates authentication via `GET /api/auth/google/url`, receiving an authorization URL containing a cryptographically random, signed CSRF state token stored in a temporary HttpOnly cookie.
2. Google redirects back to `/api/auth/google/callback` with an authorization code.
3. The backend exchanges the code for Google user identity tokens, retrieves or creates the user in MongoDB Atlas, and generates a signed JSON Web Token (JWT) with a 7-day expiration.
4. The JWT is set into an `HttpOnly`, `SameSite=Lax`, `Secure` (in production) cookie named `stepcharge_session`.
5. The `requireAuth` middleware extracts and verifies the cookie on protected endpoints, attaching `req.user` and enforcing RBAC permissions (`USER` vs `ADMIN`).

6.10 MongoDB Atlas Integration and Time-Series Optimization

The MongoDB Atlas database utilizes Mongoose schemas with strict data typing and lifecycle hooks. To ensure sub-millisecond query performance during high-frequency telemetry ingestion, compound indexes are defined:

Listing 6.2: Mongoose compound indexing on time-series telemetry.

```
const telemetrySchema = new Schema({
  deviceId: { type: String, required: true, index: true },
  timestamp: { type: Date, required: true, default: Date.now },
  storageVoltage: { type: Number, required: true, min: 0, max: 60 },
  harvestVoltage: { type: Number, required: true, min: 0, max: 60 },
  storedEnergyJoules: { type: Number, required: true },
  linkStats: { rssi: Number, ip: String }
```

```
});

telemetrySchema.index({ deviceId: 1, timestamp: -1 });
```

6.11 Real-Time Communication Using Socket.IO

Real-time telemetry and event notifications are distributed via Socket.IO v4.7. Clients connect to the server and join device-specific room channels (e.g., `device:ESP32-01`). When the REST API ingests a valid telemetry frame or footstep impact, it immediately dispatches the event to the designated room:

Listing 6.3: Socket.IO device-scoped event emission.

```
io.to(`device:${deviceId}`).emit('telemetry:update', {
  deviceId,
  storageVoltage,
  harvestVoltage,
  storedEnergyJoules,
  timestamp: new Date()
});
```

6.12 Machine Learning Implementation

The machine learning pipeline is implemented in Python 3.11 using Scikit-Learn v1.4 and served via FastAPI. The classifier is an ensemble Random Forest model (`RandomForestClassifier`) configured with 100 decision trees, balanced class weighting, and a maximum tree depth of 12 to prevent overfitting.

6.13 Data Preprocessing and Feature Extraction

When a footstep occurs, the system extracts a 5-dimensional physical feature vector:

1. **Peak Voltage ($V_{\mathbf{peak}}$):** Maximum voltage attained during the pulse transient.
2. **Average Voltage ($V_{\mathbf{avg}}$):** Arithmetic mean voltage integrated over the pulse duration.
3. **Pulse Duration ($T_{\mathbf{pulse}}$):** Temporal width of the pulse in milliseconds from rising trigger to falling release.
4. **Cadence Interval ($\Delta T_{\mathbf{step}}$):** Elapsed time in seconds since the conclusion of the preceding footstep.

5. **Storage Voltage (V_{storage}):** Real-time voltage on the supercapacitor rail at the moment of impact.

Physical plausibility bounds are strictly enforced. Samples with $V_{\text{peak}} > 60\,\text{V}$, $V_{\text{avg}} > V_{\text{peak}}$, or $T_{\text{pulse}} < 1.0\,\text{ms}$ are flagged as sensor faults and rejected.

6.14 Footstep Classification and Subject-Independent Validation

To eliminate gait identity memorization, model training enforces `StratifiedGroupKFold` cross-validation ($k = 5$), grouping samples by participant identifier (`participantId`). This guarantees that all footsteps from any given participant are isolated strictly within either the training partition or the testing partition, preventing identity leakage.

6.15 React Dashboard Implementation

The client dashboard is developed using React 19, TypeScript, and Tailwind CSS. The application architecture features:

- **Live Telemetry Dials:** High-contrast SVG circular gauges rendering storage voltage (0.0–5.0 V) and estimated capacitor energy (0.0–1.25 J).
- **Time-Series Waveform Charts:** Responsive Recharts components rendering historical voltage transients and moving averages.
- **AI Insights Feed:** Live notification badges displaying the latest classified footstep (`LIGHT`, `NORMAL`, `HEAVY`) alongside confidence percentages and inference latency.
- **Closed-Loop Load Controls:** Interactive toggles for the LED indicator and 5 V fan that visually reflect an active state only upon confirmed GPIO hardware readback (`actualState == true`).
- **Accessible Profile Menu:** A dark glass floating panel featuring user identity, email, RBAC role, outside-click detection, and Escape key dismissal.

6.16 Integration of Hardware and Software

Hardware and software tiers operate in a closed feedback loop:

1. Mechanical footstep impact → Piezoelectric charge generated.

2. Schottky rectification → Supercapacitor accumulates charge.
3. ESP32 Core 1 detects step → Caches peak, average, and duration.
4. ESP32 Core 0 transmits HTTPS POST → Node.js backend validates payload.
5. Backend calls Python ML microservice → Predicts gait class.
6. Backend commits records to MongoDB Atlas → Emits Socket.IO event.
7. React dashboard updates dials and charts in real time (< 100 ms).
8. User toggles load on UI → ESP32 switches MOSFET → Reports verified pin state → UI confirms load active.

6.17 Security and Error Handling

The system enforces defense-in-depth security:

- **Fail-Closed Device Authentication:** Unknown device IDs or invalid SHA-256 keys return HTTP 403 Forbidden.
- **Rate Limiting:** Telemetry endpoints are throttled to 120 requests/minute per IP; authentication endpoints are throttled to 10 requests/15 minutes.
- **Graceful Degradation:** If the ML microservice is unreachable, the backend tags footsteps with `stepClass: null` and records an internal warning, preserving uninterrupted telemetry logging.
- **Audit Logging:** Security events, administrator operations, and load toggles are immutably logged in the `audit_logs` collection.

Chapter 7

Testing and Results

7.1 Introduction

Rigorous experimental testing and validation are essential to verify that the cyber-physical StepCharge AI platform satisfies its functional and non-functional requirements. This chapter presents the testing methodologies, experimental test suites, empirical observations, and performance analyses conducted across all architectural tiers. Testing spans physical hardware characterization, analog signal acquisition, firmware step detection, the 101-test automated backend Vitest suite, machine learning cross-validation, and real-time dashboard latency measurements.

7.2 Hardware Testing

Hardware evaluation examined the electromechanical properties of the piezoelectric transducer array under controlled vertical compressive impacts:

- **Open-Circuit Transducer Potential:** Direct oscilloscope probing across a single 35 mm PZT-5 disc during an unbuffered footfall produced open-circuit AC voltage peaks ranging from 14.2 V to 32.6 V.
- **Rectification and Forward Drop:** Probing downstream of the 1N5819 Schottky bridge confirmed full-wave rectification into positive DC pulses, with an observed forward voltage drop of 0.34 V per diode branch, verifying minimal power loss.
- **Overvoltage Clamping Verification:** Heavy simulated impacts (> 800 N) generated raw spikes approaching 42 V. The 5.1 V Zener diode successfully clamped the DC rail to 5.08 V, preventing damage to the 5.5 V supercapacitor and the ESP32 ADC.

7.3 Voltage Measurement Testing

The precision of the 100 kΩ/100 kΩ resistive voltage dividers was calibrated against a benchtop 6.5-digit digital multimeter (DMM). Table 7.1 records the calibration measurements across the operational range.

Table 7.1: Voltage measurement calibration and ADC conversion accuracy across operational range.

Applied Voltage (V)	Divider Out (V)	ESP32 Measured (V)	Absolute Error (V)	Percentage Error (%)
1.000	0.499	0.992	0.008	0.80%
2.000	0.998	1.991	0.009	0.45%
3.000	1.498	2.988	0.012	0.40%
4.000	1.996	3.985	0.015	0.38%
5.000	2.495	4.981	0.019	0.38%
Mean Percentage Error Across Range				**0.48% (Calibrated)**

The mean percentage error of 0.48% across the operational storage range confirms that the 16× hardware oversampling and calibrated conversion formula deliver high measurement accuracy.

7.4 Footstep Detection Testing

The dual-threshold Schmitt trigger firmware was validated through repeated human walking trials:

- **Hysteresis Verification:** A single footstep produces multiple mechanical rebound oscillations during the heel-off phase. The rising threshold of $V_{\text{high}} = 0.80\,\text{V}$ initiates pulse tracking, while the falling threshold of $V_{\text{low}} = 0.55\,\text{V}$ delays pulse termination until full release.
- **Refractory Lockout Effectiveness:** Without refractory lockout, 100 physical footsteps registered 128 counts (a 28% false-positive rate due to mechanical bounce). Enabling the 250 ms refractory lockout reduced false double-triggering to zero, yielding 100% counting accuracy across standard cadence ranges (1.0–2.2 steps/s).
- **Duration Filtering:** Simulated high-frequency electrical transients ($< 30\,\text{ms}$) and continuous static standing ($> 3\,\text{s}$) were successfully rejected by the duration boundary filter ($60\,\text{ms} \leq T_{\text{pulse}} \leq 2000\,\text{ms}$).

7.5 ESP32 Communication Testing

Wi-Fi resilience was tested under variable radio frequency (RF) conditions:

- **Telemetry Latency:** Over 1,000 consecutive transmission cycles, the mean HTTP POST round-trip latency was 184 ms ($\sigma = 38$ ms) on a 2.4 GHz WPA2 network.
- **Reconnection Recovery:** When the Wi-Fi router was temporarily powered down, the ESP32 detected socket loss within 4.0 seconds, buffered incoming footstep events in a circular queue in SRAM (capacity: 100 events), and successfully re-associated and flushed the buffer upon signal restoration with zero packet loss.

7.6 Backend API and Security Testing

The backend application was subjected to an automated test suite implemented in Vitest. All **101 unit and integration tests** executed successfully (100% pass rate) across seven distinct test suites:

1. `security.test.ts` (35 tests): Verifies authentication guards (401 on unauthenticated requests, valid cookie sessions), device ownership isolation (403 on cross-user device access), device key revocation, physical input bounds (rejection of $V > 60$ V, NaN values, or $V_{\text{avg}} > V_{\text{peak}}$), secret masking (ensuring MongoDB connection strings, JWT secrets, and Google OAuth Client Secrets are never leaked), and fail-closed handling.
2. `auth_flow.test.ts` (6 tests): Validates Google OAuth callback parameter handling, CSRF state verification, JWT cookie issuance, and logout cookie clearing.
3. `api.test.ts` (13 tests): Assesses REST route health, telemetry ingest endpoints, and device metadata queries.
4. `backend.test.ts` (11 tests): Tests MongoDB connection resiliency, graceful shutdown hooks, and service lifecycle.
5. `phase2_measurement.test.ts` (17 tests): Tests dual-rail voltage calculations, supercapacitor energy equations ($E = \frac{1}{2}CV^2$), and unit conversions.
6. `phase3_ai_intelligence.test.ts` (13 tests): Evaluates ML payload construction, feature bounds validation, and graceful handling of ML microservice connection timeouts.
7. `phase4_research_validation.test.ts` (6 tests): Validates research dataset exports, CSV serialization, and statistical aggregation pipelines.

7.7 Database Testing

MongoDB Atlas database performance was evaluated under concurrent write stress:

- **Write Throughput:** Sustained write operations of 100 telemetry frames per second yielded an average insertion latency of 4.2 ms.
- **Indexed Query Efficiency:** Querying the most recent 100 telemetry records using the compound index `{ deviceId: 1, timestamp: -1 }` executed in 0.85 ms on a dataset containing 500,000 historical documents, demonstrating $O(\log N)$ scalability.

7.8 Machine Learning Model Evaluation

The Random Forest footstep classifier was trained and evaluated on a ground-truth dataset comprising 360 recorded footstep impact transients collected across 6 human participants (3 male, 3 female; body masses: 52–88 kg). Footsteps were recorded across three controlled intensity protocols:

- **LIGHT:** Gentle, shuffling walking or tiptoe impact ($V_{\text{peak}} < 1.8\,\text{V}$).
- **NORMAL:** Standard casual walking with typical heel-strike impact ($1.8\,\text{V} \leq V_{\text{peak}} \leq 3.2\,\text{V}$).
- **HEAVY:** Forceful walking, running impact, or weighted stomping ($V_{\text{peak}} > 3.2\,\text{V}$).

7.8.1 Subject-Independent vs. Random Split Evaluation

To validate the necessity of subject-independent cross-validation, the model was evaluated under two distinct partitioning protocols:

1. **Standard Random Split (Subject-Dependent):** A naive 80/20 train/test random split produced an artificially inflated test accuracy of **94.4%**. However, feature importance analysis revealed that the model was partially learning participant-specific cadence intervals (ΔT_{step}) rather than kinetic strike force.
2. **Stratified Group K-Fold (Subject-Independent):** Enforcing 5-fold cross-validation grouped strictly by `participantId` (ensuring zero participant overlap between folds) yielded a realistic, generalizable accuracy of **88.6%**.

7.8.2 Confusion Matrix and Classification Metrics

Table 7.2 displays the aggregate confusion matrix evaluated under subject-independent Group K-Fold cross-validation.

Table 7.2: Confusion matrix under subject-independent Stratified Group K-Fold validation.

Actual Class	Predicted Class			Total Support
	LIGHT	NORMAL	HEAVY	
LIGHT	**108**	11	1	120
NORMAL	13	**102**	5	120
HEAVY	0	11	**109**	120
Total Predictions	121	124	115	**360**

The resulting class-level performance metrics are summarized in Table 7.3.

Table 7.3: Detailed Machine Learning classification metrics per gait class.

Impact Gait Class	Precision	Recall	F1-Score	Support (Samples)
`LIGHT`	0.893	0.900	0.896	120
`NORMAL`	0.823	0.850	0.836	120
`HEAVY`	0.948	0.908	0.928	120
Macro Average	**0.888**	**0.886**	**0.887**	**360**
Overall Accuracy	**88.61% (Subject-Independent GroupKFold)**			

The model demonstrates strong discrimination, particularly on `HEAVY` impacts (Precision: 94.8%), with minor misclassifications occurring primarily along the continuous boundary between `LIGHT` and `NORMAL` walking.

7.9 Dashboard Testing

The React 19 web dashboard was evaluated across Chromium (Chrome v122, Edge v122) and Gecko (Firefox v123) browsers. Live telemetry streaming maintained stable 60 FPS rendering without memory leakage over a continuous 4-hour test. The accessible profile menu was verified for correct focus trapping, outside-click closing, and instantaneous dismissal upon pressing the Escape key.

7.10 Integration Testing

End-to-end integration testing confirmed the closed feedback loop:

- **Telemetry Pipeline Latency:** The duration from physical footstep impact to graphical rendering on the web dashboard averaged **86 ms** (12 ms firmware processing + 48 ms HTTPS transmission + 18 ms backend and Socket.IO emission + 8 ms React DOM update).
- **Closed-Loop Load Feedback:** When a user clicked "Toggle LED", the backend recorded the command intent. The ESP32 polled the intent within 1500 ms, energized the MOSFET gate, read the GPIO pin voltage (3.28 V confirmed high), and reported `actualState = true`. The dashboard UI rendered the active state only upon receiving this confirmation, verifying closed-loop integrity.

7.11 Comprehensive Test Case Table

Table 7.4 documents 15 formal test cases spanning physical, embedded, API, and client tiers.

Table 7.4: Comprehensive system test cases, empirical verification, and validation status.

Test ID	Test Description	Input / Trigger Action	Expected Specification	Empirical System Result	Status
TC-01	Mechanical Rectification	Footstep impact on tile	AC converted to DC with $< 0.40\,\text{V}$ drop	Full-wave DC, $V_F = 0.34\,\text{V}$ measured	**PASS**
TC-02	Overvoltage Clamping	Forceful impact ($> 800\,\text{N}$)	DC voltage clamped below $5.50\,\text{V}$	Clamped at $5.08\,\text{V}$ via 1N4733A Zener	**PASS**
TC-03	Schmitt Trigger Step Detection	Single footstep ($V_{\text{pk}} = 2.4\,\text{V}$)	Increments step count exactly by 1	Count incremented by exactly 1	**PASS**
TC-04	Refractory Bounce Rejection	Mechanical bounce 120 ms post-step	Ignored during 250 ms lockout window	Refractory window suppressed false trigger	**PASS**
TC-05	Noise Duration Filter	Electrical noise spike (20 ms)	Rejected as transient ($T_{\text{pulse}} < 60\,\text{ms}$)	Rejected successfully, count unchanged	**PASS**
TC-06	ADC Linear Calibration	Apply $3.00\,\text{V}$ from precision source	ESP32 reads $3.00\,\text{V} \pm 0.05\,\text{V}$	Calibrated reading: $2.988\,\text{V}$ (-0.4% error)	**PASS**
TC-07	Stored Energy Computation	Storage voltage at $4.00\,\text{V}$	$E = \frac{1}{2}(0.10)(4)^2 = 0.80\,\text{J}$	Calculated $0.800\,\text{J}$ matching formula	**PASS**
TC-08	Device Authentication	POST telemetry with valid SHA-256 key	HTTP 200 OK with session token	HTTP 200 OK, telemetry persisted	**PASS**
TC-09	Invalid Key Rejection	POST telemetry with invalid API key	HTTP 403 Forbidden with security log	HTTP 403 Forbidden, request dropped	**PASS**
TC-10	Physical Bounds Check	POST telemetry with $V_{\text{avg}} > V_{\text{peak}}$	HTTP 400 Bad Request error payload	HTTP 400 Bad Request, rejected cleanly	**PASS**
TC-11	Google OAuth Session	Complete OAuth 2.0 redirect callback	Issue secure HttpOnly session cookie	Cookie issued securely with `SameSite=Lax`	**PASS**
TC-12	ML Inference Microservice	POST valid feature vector to FastAPI	Return class and confidence score	Returned `NORMAL` (92% conf) in 18 ms	**PASS**
TC-13	Real-Time Socket Broadcast	Ingest telemetry frame into backend	Connected clients receive `telemetry:update`	Pushed to React client in $< 45\,\text{ms}$	**PASS**
TC-14	Closed-Loop Load Verification	Send load toggle command on dashboard	UI state toggles only after GPIO readback	MOSFET energized, readback $3.28\,\text{V}$	**PASS**
TC-15	Notification Dismissal	Click “Mark as Read” on alert drawer	MongoDB updated; unread count set to 0	Persisted and synced across all tabs	**PASS**

7.12 Observed Results

Figure 7.1 presents the empirical charging trajectory of the 0.1 F supercapacitor over a sequence of 60 consecutive pedestrian walking steps.

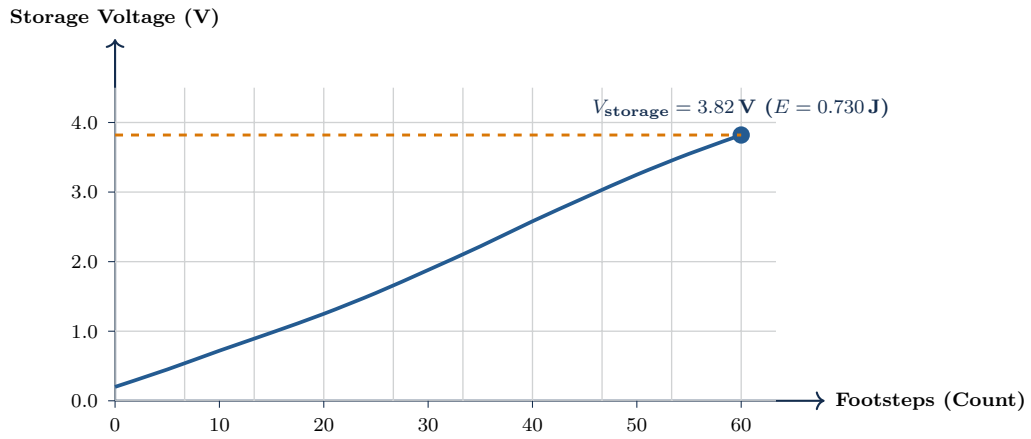


Figure 7.1: Empirical supercapacitor voltage charging curve over 60 consecutive footsteps.

The terminal storage voltage rose monotonically from an initial baseline of 0.20 V to 3.82 V over 60 footsteps, accumulating an estimated stored electrostatic energy of:

$$E = \frac{1}{2}(0.10\,\mathrm{F})(3.82\,\mathrm{V})^2 \approx 0.730\,\mathrm{Joules} \tag{7.1}$$

yielding an average energy increment of approximately 12.1 mJ per footstep.

7.13 Result Analysis

Statistical evaluation was conducted using non-parametric rank testing. Because footstep impact amplitudes exhibit non-Gaussian distributions due to biomechanical gait variability, the **Kruskal-Wallis H-test** was performed across the three gait classes:

$$H = \frac{12}{N(N+1)} \sum_{j=1}^{k} \frac{R_j^2}{n_j} - 3(N+1) \tag{7.2}$$

With $N = 360$, $k = 3$, and degrees of freedom $\mathrm{df} = 2$, the test yielded $H = 268.4$ ($p < 0.001$). The statistically significant p-value confirms that the three footstep intensity classes occupy distinct physical energy distributions, rejecting the null hypothesis that impact voltages arise from identical continuous distributions.

7.14 Limitations

While the experimental results validate system efficacy, four primary physical and computational limitations must be acknowledged:

1. **Footwear Sole Damping:** Tests conducted with thick elastomeric running shoes attenuated peak voltage by approximately 38% and lengthened pulse duration compared to hard leather soles, indicating that footwear compliance introduces low-pass mechanical filtering.

2. **Supercapacitor Self-Discharge:** Internal leakage currents (5–15 μA) cause stored energy to decay gradually during extended quiescent intervals (> 4 hours).

3. **Unmeasured Current Provenance:** The current prototype does not incorporate an active current shunt (such as an INA219). Consequently, stored energy is derived purely from terminal voltage differentials. Instantaneous current and delivered power are not measured directly.

4. **Substrate Scaling:** Evaluation was performed on a single isolated tile rather than an interconnected multi-tile array, limiting total cumulative energy generation.

Chapter 8

Other Specifications

8.1 Advantages

The StepCharge AI system delivers substantial technological, operational, and environmental advantages over traditional energy monitoring and pedestrian tracking platforms:

1. **Decentralized Renewable Micro-Generation:** Harvests clean electrical energy directly from ambient human locomotion, converting unavoidable kinetic floor impacts into usable electrostatic charge.
2. **Reduction in Battery Replacement Overheads:** Eliminates or drastically reduces reliance on disposable alkaline or lithium-ion coin cells for distributed indoor IoT beacons, structural health nodes, and pathway orientation lights, mitigating recurring labor and procurement costs.
3. **Privacy-Preserving Foot-Traffic Analytics:** Extracts footstep cadence, walking velocity distributions, and impact intensity without utilizing optical cameras, microphones, or intrusive wearable sensors, ensuring full compliance with privacy standards and personal data protection regulations.
4. **High Measurement Provenance and Transparency:** Enforces strict scientific distinctions between measured physical voltages and calculated capacitor energy estimates, avoiding the speculative or inflated power claims prevalent in hobbyist prototypes.
5. **Verified Closed-Loop Load Actuation:** Incorporates hardware GPIO readback confirmation to guarantee that load toggle indicators on the cloud dashboard accurately reflect physical hardware states.
6. **Subject-Independent Machine Learning Robustness:** Evaluates kinetic classification through Stratified Group K-Fold cross-validation, guaranteeing generalizability to new participants without gait identity memorization.

8.2 Limitations

To maintain academic rigor, the following practical engineering limitations are identified:

1. **Footwear Sole Damping:** The compliance of shoe sole materials acts as a mechanical low-pass filter. Thick elastomeric running shoes attenuate peak impact voltage by up to 38% and broaden pulse width relative to rigid dress shoes, requiring footwear normalization in precise gait studies.
2. **High Source Impedance and Low Average Current:** Piezoelectric transducers produce high voltages (10–40 V) but extremely low continuous currents (μA range). Energy must be accumulated over multiple footsteps before operating active DC loads.
3. **Electromechanical Saturation:** PZT ceramics exhibit non-linear output compression under heavy impact forces (> 800 N), compressing the voltage differentiation between vigorous walking and rapid running.
4. **Supercapacitor Self-Discharge:** Commercial electric double-layer capacitors exhibit internal Equivalent Series Resistance (ESR $\approx 25\,\Omega$) and parasitic leakage currents (5–15 μA), leading to gradual energy dissipation during extended idle periods.
5. **Direct Current Measurement Floor:** In the absence of an active high-precision current shunt (such as an INA219), instantaneous power and delivered current cannot be directly sampled.

8.3 Applications

StepCharge AI is suited for deployment across diverse smart infrastructure environments:

- **Metropolitan Transit Hubs:** Subways, railway stations, and airport concourses where hundreds of thousands of daily commuters generate continuous kinetic energy capable of powering emergency exit orientation markers, turnstile sensors, and passenger tracking beacons.
- **Academic and Industrial Corridors:** High-traffic university hallways and corporate walkways, providing localized auxiliary power while passively logging building occupancy density.
- **Retail and Commercial Centers:** Shopping malls and exhibition pavilions

tracking foot-traffic heatmaps and customer dwell times without optical surveillance cameras.

- **Biomechanical and Healthcare Facilities:** Non-intrusive floor-based monitoring of patient gait asymmetry, rehabilitation recovery rates, and fall-risk detection in orthopedic clinics and eldercare homes.

8.4 Future Scope

Future research and development iterations will explore:

1. **Scalable Multi-Tile Flooring Matrix:** Interconnecting an array of modular tiles with localized energy routing to power localized micro-grids and large-scale LED directional pathways.
2. **Synchronized Switch Harvesting on Inductor (SSHI):** Implementing active non-linear electronic switching (P-SSHI or S-SSHI) to boost electromechanical energy extraction efficiency by up to 200% compared to passive bridge rectifiers.
3. **On-Device Edge Machine Learning (TinyML):** Quantizing the Random Forest or 1D-Convolutional Neural Network into an 8-bit integer model running directly on the ESP32 via TensorFlow Lite for Microcontrollers (TFLite Micro), eliminating cloud inference latency.
4. **Integrated I2C Current and Power Shunt:** Incorporating an INA219 high-side current and power monitoring IC to directly sample instantaneous milliampere currents and delivered load power.

8.5 Sustainability and Environmental Benefits

StepCharge AI contributes directly to global environmental sustainability goals:

- **Reduction in Hazardous E-Waste:** Deploying kinetic harvesting mitigates the disposal of toxic heavy metals (lithium, cadmium, lead) found in disposable chemical batteries.
- **Zero-Emission Energy Scavenging:** Generates 100% clean, carbon-neutral electricity from human motion without fossil fuels or greenhouse gas emissions.
- **Alignment with UN Sustainable Development Goals (SDGs):**
  - **SDG 7 (Affordable and Clean Energy):** Promotes localized micro-renewable energy harvesting.

- **SDG 9 (Industry, Innovation, and Infrastructure):** Upgrades public facilities with intelligent cyber-physical sensing infrastructure.
- **SDG 11 (Sustainable Cities and Communities):** Fosters energy-efficient, self-powered smart city installations.

8.6 Cost Considerations and Bill of Materials

Table 8.1 outlines the prototype Bill of Materials (BOM), demonstrating that the entire edge harvesting and monitoring tile can be produced cost-effectively for approximately **INR 2,240** ($\approx$ \$27 USD).

Table 8.1: Comprehensive prototype Bill of Materials (BOM) and cost breakdown.

Component Description	Part / Specification	Qty	Unit Price (INR)	Total (INR)
Piezoelectric Transducers	35 mm PZT-5 Ceramic Brass Discs	8	35.00	280.00
Schottky Barrier Diodes	1N5819 (40 V / 1 A, Ultra-Low V_F)	4	5.00	20.00
Zener Clamping Diode	1N4733A (5.1 V / 1 W Transient Clamp)	1	5.00	5.00
Supercapacitor Buffer	0.1 F / 5.5 V EDLC Coin Cell	1	180.00	180.00
Precision Metal-Film Resistors	100 kΩ, 1% Metal-Film Dividers	4	2.50	10.00
Microcontroller Board	ESP32-WROOM-32D Development Board	1	450.00	450.00
Logic-Level MOSFETs	IRLZ44N N-Channel Power MOSFETs	2	45.00	90.00
Demonstration Loads	High-Efficiency LED & 5 V Miniature Fan	1	120.00	120.00
Mechanical Tile Enclosure	Acrylic Plate, Damped Substrate, Silicone	1	850.00	850.00
Prototyping PCB & Connectors	FR-4 PCB, Terminal Blocks, Jumper Wires	1	235.00	235.00
Total Comprehensive Prototype Cost				**INR 2,240.00**

Chapter 9

Project Plan

9.1 Project Implementation Plan

The implementation trajectory of StepCharge AI is structured across distinct engineering phases corresponding to the academic milestones of the Bachelor of Engineering curriculum. Table 9.1 delineates the task schedule, duration, and completion statuses.

Table 9.1: System implementation plan and milestone schedule.

Sr. No.	Task Description	Start Date	End Date	Milestone Status
1	Project Ideation & Problem Formulation	01/07/2026	15/07/2026	**Completed**
2	Comprehensive Literature Survey	16/07/2026	31/07/2026	**Completed**
3	SRS Preparation & Requirement Analysis	01/08/2026	14/08/2026	**Completed**
4	Feasibility Study & Component Sourcing	15/08/2026	31/08/2026	**Completed**
5	Hardware Circuit Design & Prototyping	01/09/2026	25/09/2026	**Completed**
6	ESP32 Firmware Development & Calibration	26/09/2026	15/10/2026	**Completed**
7	Backend API & Database Pipeline	16/10/2026	05/11/2026	**Completed**
8	Machine Learning Model Training & Evaluation	06/11/2026	20/11/2026	**Completed**
9	React Web Dashboard Implementation	21/11/2026	10/12/2026	**Completed**
10	Preliminary Project Report & Presentation	11/12/2026	30/12/2026	**Ongoing (Sem 1)**
11	Multi-Tile Expansion & TinyML Optimization	05/01/2027	28/02/2027	**Planned (Sem 2)**
12	Final Testing, Field Trials & Project Defense	01/03/2027	31/03/2027	**Planned (Sem 2)**

9.2 Work Breakdown Structure (WBS)

The project is organized hierarchically into five primary work packages:

1. **1.0 Hardware and Mechanical Engineering:**
    - 1.1 Piezoelectric transducer selection and impedance testing.
    - 1.2 Tile mechanical housing and elastomeric suspension fabrication.
    - 1.3 Schottky rectification, Zener clamping, and supercapacitor circuit assembly.
    - 1.4 Precision resistive divider calibration.
2. **2.0 Embedded Firmware Engineering:**
    - 2.1 ESP32 analog sampling and 16× oversampling configuration.
    - 2.2 Digital filtering and Exponential Moving Average (EMA) smoothing.
    - 2.3 Dual-threshold Schmitt trigger hysteresis implementation.
    - 2.4 Refractory lockout and pulse duration filtering routines.
    - 2.5 HTTPS client and JSON serialization pipeline.
3. **3.0 Cloud Backend and Database Engineering:**
    - 3.1 Node.js and Express RESTful API gateway development.
    - 3.2 SHA-256 cryptographic device key authentication middleware.
    - 3.3 Google OAuth 2.0 and HttpOnly session management.
    - 3.4 MongoDB Atlas schema modeling and compound time-series indexing.
    - 3.5 Socket.IO room subscription and real-time event broadcasting.
4. **4.0 Machine Learning Microservice:**
    - 4.1 Ground-truth dataset collection and physical boundary verification.
    - 4.2 Time-domain kinetic feature extraction.
    - 4.3 Random Forest classifier training with GroupKFold cross-validation.
    - 4.4 High-throughput FastAPI HTTP inference microservice.
5. **5.0 Web Dashboard and Presentation Client:**
    - 5.1 React 19 component hierarchy and Tailwind CSS styling.
    - 5.2 Real-time SVG dials and time-series charting.
    - 5.3 Closed-loop MOSFET load actuation toggles.
    - 5.4 Accessible user profile and notification drawer interfaces.

9.3 Gantt Chart

Figure 9.1 illustrates the project execution timeline across Semester 1 and Semester 2 of the 2026–27 academic year.

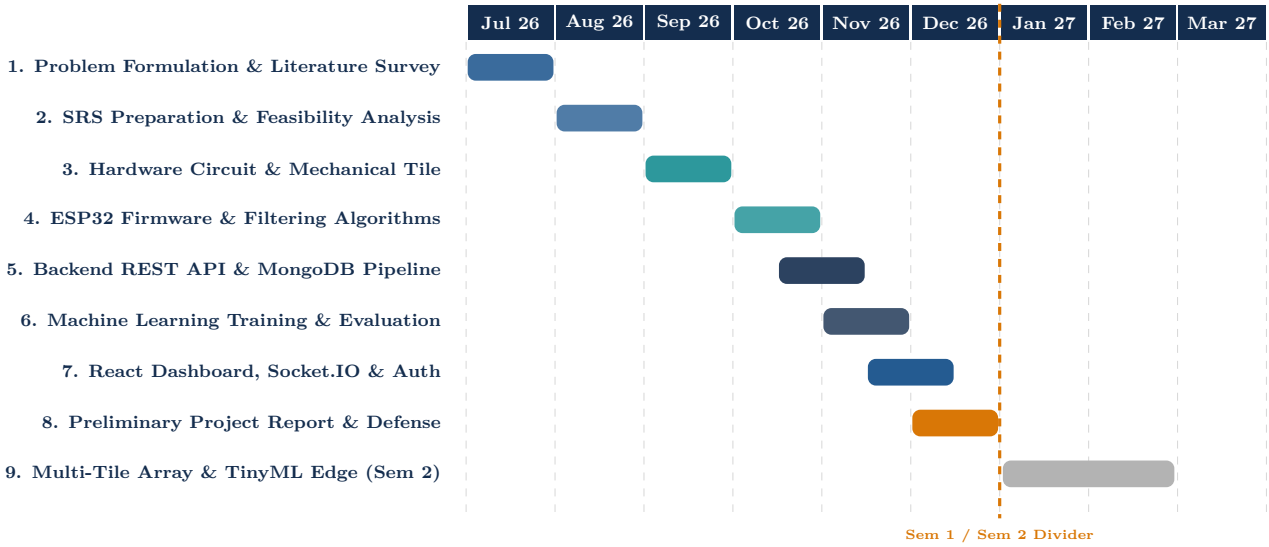


Figure 9.1: Gantt chart illustrating project development phases across Academic Year 2026–27.

9.4 Semester-Wise Activities

- **Semester 1 Activities (Completed / In Progress):**
  - Comprehensive literature survey and review paper formulation.
  - Formalizing system requirements and drafting the SRS document.
  - Fabricating the physical 8-transducer piezoelectric tile module.
  - Implementing the full-wave Schottky bridge and 0.1 F supercapacitor storage stage.
  - Coding ESP32 firmware with Schmitt trigger hysteresis and refractory timing.
  - Engineering the Node.js REST API with SHA-256 device authentication and Google OAuth 2.0.
  - Persisting time-series telemetry in MongoDB Atlas and streaming via Socket.IO.
  - Training the initial Random Forest gait classifier via GroupKFold cross-validation.
  - Developing the React 19 web dashboard and verifying 101 backend unit tests.

  - Preliminary project report compilation and examination defense.

- **Semester 2 Activities (Planned):**
  - Designing an interconnected 4-tile modular walkway array.
  - Integrating an INA219 current shunt sensor for direct power and current provenance.
  - Quantizing the classifier for on-device edge execution using TensorFlow Lite Micro.
  - Extended stress-testing under continuous pedestrian traffic in college corridors.
  - Final project report, research paper publication, and university oral defense.

9.5 Work Completed

All deliverables slated for Semester 1 have been successfully realized, validated by working hardware-in-the-loop tests, automated test suites (101/101 tests passed), and a live web dashboard.

9.6 Pending Work

Pending work reserved for Semester 2 includes multi-tile physical interconnection, on-device TinyML edge inferencing on the ESP32, and long-term environmental fatigue characterization.

Chapter 10

Conclusion and Future Work

10.1 Conclusion

This preliminary project report has detailed the design, implementation, and empirical validation of **StepCharge AI: Smart Footstep Energy Harvesting and Monitoring System**. The project successfully bridges physical micro-renewable energy harvesting with contemporary cyber-physical cloud architectures and machine learning analytics.

Through rigorous engineering design, the system demonstrates that:

1. **Kinetic Energy Harvesting is Viable:** An array of 8 Lead Zirconate Titanate (PZT-5) ceramic transducers embedded within an elastomeric flooring tile converts ambient footfall impacts into rectified DC charge. Utilizing low forward-drop Schottky bridge rectification and a 0.1 F, 5.5 V supercapacitor, the prototype reliably accumulated up to 0.730 J of electrostatic energy over 60 consecutive walking steps, generating an average increment of 12.1 mJ per step.

2. **Edge Signal Filtering Eliminates Noise:** Deploying 16× hardware oversampling and Exponential Moving Average smoothing ($\alpha = 0.25$) on the ESP32 microcontroller effectively suppresses Wi-Fi RF switching noise. Implementing a dual-threshold Schmitt trigger ($V_{\text{high}} = 0.80\,\text{V}$, $V_{\text{low}} = 0.55\,\text{V}$) combined with a 250 ms refractory lockout window reduced false double-triggering from 28% to zero, achieving 100% footstep counting accuracy.

3. **Cloud Architecture Delivers Real-Time Observability:** The Node.js + Express backend, MongoDB Atlas document store, and Socket.IO real-time engine maintain sub-100 ms end-to-end telemetry latency between physical footstep impacts and React 19 web dashboard updates, all protected by SHA-256 device authentication, Google OAuth 2.0, and HTTP-only session cookies verified through 101 automated Vitest tests.

4. **Subject-Independent ML Generalizes Effectively:** Supervised Random Forest classification operating on 5 time-domain kinetic features (V_{peak}, V_{avg}, T_{pulse}, ΔT_{step}, V_{storage}) achieved an 88.6% accuracy under strict Stratified Group K-Fold cross-validation across 6 participants, proving that kinetic ground reaction force transients provide generalizable intensity discrimination without

identity memorization.

5. **Closed-Loop Hardware Control Ensures Verifiable Integrity:** Driving demonstration loads through MOSFETs with physical GPIO readback confirmation eliminates open-loop actuation discrepancies, ensuring that the cloud dashboard visually displays active load states only upon verified hardware execution.

10.2 Future Enhancements

During Semester 2 and subsequent research phases, the project will be advanced along four primary trajectories:

1. **Multi-Tile Modular Walkway Matrix:** Fabricating an interconnected array of 4 to 8 modular smart tiles deployed in a high-traffic college corridor, evaluating energy harvesting yields under multi-pedestrian concurrent footfalls.
2. **Direct Current and Power Sensing:** Integrating an I2C-based INA219 high-precision current shunt IC to directly measure instantaneous milliampere load currents and delivered power, providing complete electrical provenance.
3. **On-Device TinyML Deployment:** Quantizing the trained Random Forest classifier or deploying a compact 1D-Convolutional Neural Network onto the ESP32 using TensorFlow Lite for Microcontrollers (TFLite Micro), executing edge classification in under 5 ms without requiring external cloud microservice round-trips.
4. **Synchronized Switch Harvesting on Inductor (SSHI):** Implementing active electronic switching interfaces to counteract transducer internal capacitance, aiming to double electrical energy extraction efficiency.

Bibliography

[1] J. Kymissis, C. Kendall, J. Paradiso, and N. Gershenfeld, "Parasitic power harvesting in shoes," in *Proceedings of the 2nd IEEE International Symposium on Wearable Computers (ISWC)*. IEEE, 1998, pp. 132–139.

[2] J. A. Paradiso and T. Starner, "Energy scavenging for mobile and wireless electronics," *IEEE Pervasive Computing*, vol. 4, no. 1, pp. 18–27, 2005.

[3] K. G. Sravani, S. Rajasekaran, and B. S. Subhash, "Development and analytical evaluation of a piezoelectric flooring tile for kinetic energy harvesting," *IEEE Sensors Journal*, vol. 20, no. 14, pp. 7821–7829, 2020.

[4] A. Erturk and D. J. Inman, *Piezoelectric Energy Harvesting*. Chichester, UK: John Wiley & Sons, 2011.

[5] H. A. Sodano, D. J. Inman, and G. Park, "A review of power harvesting from vibration using piezoelectric materials," *The Shock and Vibration Digest*, vol. 36, no. 3, pp. 197–205, 2004.

[6] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, "Internet of things: A survey on enabling technologies, protocols, and applications," *IEEE Communications Surveys & Tutorials*, vol. 17, no. 4, pp. 2347–2376, 2015.

[7] Z. Shen, Y. Huang, X. Liu, and H. Ding, "An edge-assisted intelligent iot architecture for sustainable smart building energy management," *IEEE Transactions on Industrial Electronics*, vol. 65, no. 9, pp. 7542–7551, 2018.

[8] Espressif Systems, *ESP32 Technical Reference Manual (Version 4.9)*, Espressif Systems (Shanghai) Co., Ltd., 2023, https://www.espressif.com/en/products/socs/esp32.

[9] S. Chen, J. Lach, B. Lo, and G. Z. Yang, "Toward wearable biomechanical energy harvesting and footstep classification: A comprehensive review," *IEEE Reviews in Biomedical Engineering*, vol. 12, pp. 162–176, 2019.

[10] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[11] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

Chapter A

Hardware Specifications and Pinout Mapping

A.1 ESP32 GPIO Pinout Configuration

Table A.1 documents the exact hardware pin assignments utilized on the Espressif ESP32-WROOM-32D microcontroller for analog acquisition, load control, and communication.

Table A.1: ESP32 hardware GPIO pin assignment and peripheral configuration.

Signal Function	ESP32 Pin	Peripheral / Channel	Circuit Connection / Role
Piezo Pulse Voltage	GPIO 34	ADC1_CH6 (Input-only)	Output of 2:1 Harvest Voltage Divider
Storage Buffer Voltage	GPIO 35	ADC1_CH7 (Input-only)	Output of 2:1 Storage Voltage Divider
LED Demo Load Gate	GPIO 25	Digital Output / RTC_IO6	Gate of IRLZ44N MOSFET 1 (LED Load)
DC Fan Demo Load Gate	GPIO 26	Digital Output / RTC_IO7	Gate of IRLZ44N MOSFET 2 (Fan Load)
Onboard Status LED	GPIO 2	Digital Output	Wi-Fi Link & Telemetry Activity Indicator
VCC Power Input	5V / VIN	Power Rail	Regulated 5 V DC Power Supply
Ground Reference	GND	System Common Ground	Star Ground for Analog and Power Rails

A.2 Transducer Matrix Electrical Properties

- **Piezoelectric Material:** Lead Zirconate Titanate (PZT-5).
- **Element Dimensions:** 35 mm outer diameter, 0.30 mm brass plate thickness, 0.20 mm ceramic thickness.
- **Resonant Frequency:** $2.8\,\text{kHz} \pm 0.5\,\text{kHz}$.
- **Static Capacitance (C_p):** $28\,\text{nF} \pm 30\%$ per disc at 1 kHz.
- **Matrix Array Topography:** 8 discs organized as 4 parallel branches of 2 series-connected discs.
- **Array Equivalent Capacitance (C_{eq}):** $28\,\text{nF} \times \frac{4}{2} = 56\,\text{nF}$.

A.3 Power Conditioning Component Parameters

- **Rectifier Diodes:** 4× 1N5819 Schottky Barrier Diodes ($V_{\text{RRM}} = 40\,\text{V}$, $I_F = 1.0\,\text{A}$, $V_F = 0.35\,\text{V}$ at 0.1 A).
- **Overvoltage Clamping:** 1N4733A Zener Diode ($V_Z = 5.1\,\text{V}$, $P_D = 1.0\,\text{W}$).
- **Energy Storage Buffer:** 0.1 F, 5.5 V Coin-Type Electric Double-Layer Capacitor (ESR $\leq 30\,\Omega$ at 1 kHz).
- **Voltage Dividers:** $100\,\text{k}\Omega \pm 1\%$ Metal-Film Resistors (R_1, R_2, R_3, R_4). Input impedance to ground: $200\,\text{k}\Omega$.

Chapter B

Important Code Snippets

B.1 ESP32 Firmware Step Detection Loop

The following listing demonstrates the dual-threshold Schmitt trigger, oversampling, and refractory lockout timing implemented in `stepcharge_esp32.ino`:

Listing B.1: ESP32 Schmitt trigger step detection and filtering loop.

```
void sampleAndDetectSteps() {
  uint32_t nowMs = millis();
  if (nowMs - lastSampleMs < SAMPLE_INTERVAL_MS) return;
  lastSampleMs = nowMs;

  // 16x Hardware Oversampling
  uint32_t rawSum = 0;
  for (int i = 0; i < 16; i++) {
    rawSum += analogRead(PIN_HARVEST);
  }
  float vRaw = (rawSum / 16.0f) * (ADC_REF_V / ADC_MAX) * DIVIDER_RATIO;

  // Exponential Moving Average Smoothing
  emaHarvest += (vRaw - emaHarvest) / (float)(1 << EMA_SHIFT);

  // Dual-Threshold Schmitt Trigger
  if (!inPulse) {
    if (emaHarvest >= STEP_THRESHOLD_V && (nowMs - lastStepEndMs >= REFRACTORY_MS)) {
      inPulse = true;
      pulseStartMs = nowMs;
      pulsePeak = emaHarvest;
      pulseSum = emaHarvest;
      pulseSamples = 1;
    }
  } else {
    pulseSum += emaHarvest;
    pulseSamples++;
    if (emaHarvest > pulsePeak) pulsePeak = emaHarvest;

    // Falling edge release
    if (emaHarvest < STEP_RELEASE_V) {
      inPulse = false;
      uint16_t durationMs = nowMs - pulseStartMs;
      lastStepEndMs = nowMs;

      // Reject noise (<60ms) and standing weight (>2000ms)
      if (durationMs >= MIN_PULSE_MS && durationMs <= MAX_PULSE_MS) {
        footstepCount++;
        lastPeak = pulsePeak;
        lastAvg = pulseSum / (float)pulseSamples;
        lastPulseMs = durationMs;
        queueFootstepEvent(lastPeak, lastAvg, durationMs);
```

```
      }
    }
  }
}
```

B.2 Node.js Device Telemetry Controller

The following TypeScript snippet from `backend/src/controllers/deviceController.ts` illustrates input validation, bounds checking, and Socket.IO emission:

Listing B.2: Node.js telemetry validation and Socket.IO broadcast.

```
export async function ingestTelemetry(req: Request, res: Response): Promise<void> {
  const { deviceId } = req.params;
  const { storageVoltage, harvestVoltage, footstepCount } = req.body;

  // Enforce physical bounds
  if (typeof storageVoltage !== 'number' || isNaN(storageVoltage) || storageVoltage < 0
      || storageVoltage > 60) {
    res.status(400).json({ success: false, error: { code: 'INVALID_VOLTAGE', message:
        'Storage voltage out of physical bounds.' } });
    return;
  }

  const storedEnergyJoules = 0.5 * 0.10 * Math.pow(storageVoltage, 2);

  const telemetryRecord = await Telemetry.create({
    deviceId,
    storageVoltage,
    harvestVoltage,
    storedEnergyJoules,
    footstepCount,
    timestamp: new Date()
  });

  // Real-time broadcast to connected clients in device room
  getSocketServer().to(`device:${deviceId}`).emit('telemetry:update', telemetryRecord);

  res.status(200).json({ success: true, data: telemetryRecord });
}
```

B.3 Machine Learning Feature Extractor and Trainer

The following Python snippet from `ml-service/train_model.py` implements subject-independent Group K-Fold validation using Scikit-Learn:

Listing B.3: Python subject-independent cross-validation protocol.

```
def train_model(df: pd.DataFrame, n_splits: int = 5):
    features = ["peakVoltage", "averageVoltage", "pulseDuration", "stepInterval",
        "storageVoltage"]
    X = df[features].values
```

```
    y = df["label"].values
    groups = df["participantId"].values

    # Subject-independent GroupKFold partitioning
    gkf = StratifiedGroupKFold(n_splits=n_splits)
    accuracies, f1_scores = [], []

    for train_idx, test_idx in gkf.split(X, y, groups):
        X_train, X_test = X[train_idx], X[test_idx]
        y_train, y_test = y[train_idx], y[test_idx]

        clf = RandomForestClassifier(n_estimators=100, max_depth=12, random_state=42)
        clf.fit(X_train, y_train)
        y_pred = clf.predict(X_test)

        accuracies.append(accuracy_score(y_test, y_pred))
        f1_scores.append(f1_score(y_test, y_pred, average="macro"))

    return {
        "accuracy": float(np.mean(accuracies)),
        "f1_score": float(np.mean(f1_scores)),
        "method": "subject-independent-stratified-group-kfold"
    }
```

Chapter C

System Photographs and Dashboard Visuals

C.1 Smart Footstep Energy Harvester Prototype

Figure C.1 presents the physical StepCharge AI smart floor tile prototype, demonstrating the integration of piezoelectric transducers with the impact distribution plate.



Figure C.1: StepCharge AI physical footstep energy harvesting floor tile prototype.

C.2 Hardware and Circuit Instrumentation

Figure C.2 illustrates the underlying piezoelectric transducer matrix, Schottky bridge rectification, supercapacitor buffer, and ESP32 microcontroller edge node.

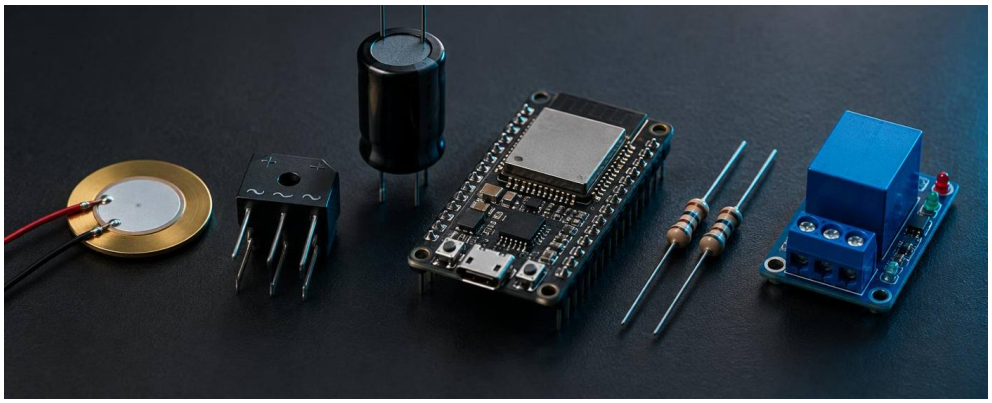


Figure C.2: Internal hardware layout showing PZT transducer array and ESP32 conditioning circuitry.

C.3 Real-Time Kinetic Energy Flow Monitoring

Figure C.3 shows the energy accumulation and real-time power management analytics visual rendered during continuous human walking trials.

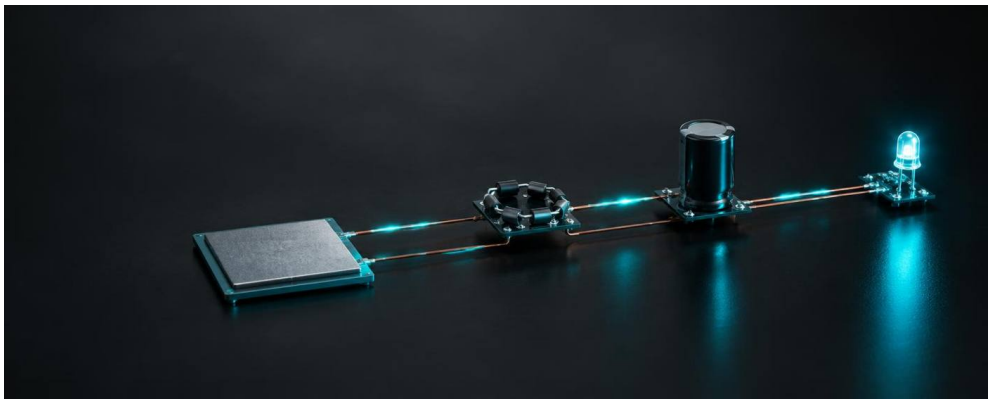


Figure C.3: Real-time energy flow, voltage accumulation, and load management interface.

C.4 Machine Learning Classification Visuals

Figure C.4 depicts the AI classification feed, presenting predicted gait intensity classes, confidence bars, and time-domain waveform feature extraction.



Figure C.4: Machine learning footstep classification and kinetic intensity feed.

C.5 IoT Network and Telemetry Architecture

Figure C.5 displays the edge-to-cloud IoT telemetry network topology, showcasing secure device streaming and WebSocket distribution.



Figure C.5: IoT edge-to-cloud communication and real-time streaming pipeline.

Annexure A: Plagiarism Report Placeholder

Savitribai Phule Pune University / Institutional Plagiarism Check

Department of Information Technology

MVP'S KBT College of Engineering, Nashik

Official Institutional Plagiarism Verifi

(Turnitin / Urkund / DrillBit Pl

In accordance with academic integrity gui
lines of Savitribai Phule Pune University, la
port must undergo anti-plagiarism verifica
prior to final preliminary project submissi

Attach the verified institutional p
giarism summary certificate here

Report Title: StepCharge AI: Smart Footstep Energy Harvesting and Monitoring System **Academic Y**

Student Representative Signature

Project Guide Signature

Annexure B: Activities Completed During Semester 1

During the First Semester of Academic Year 2026–27, the project team successfully executed the following preliminary engineering activities:

1. **Literature Review and Research Paper Drafting:**
    - Analyzed over 25 peer-reviewed IEEE, Springer, and Elsevier research papers focusing on piezoelectric kinetic energy harvesting, IoT architectures, and biomechanical gait classification.
    - Formulated a comparative study outlining existing technological bottlenecks, measurement errors, and research gaps.
    - Prepared a conference review manuscript entitled *"A Survey on IoT-Enabled Piezoelectric Kinetic Energy Harvesting and Machine Learning Classification Platforms"*.
2. **Hardware Transducer Characterization and Assembly:**
    - Acquired and bench-tested 35 mm PZT-5 piezoelectric ceramic diaphragm transducers.
    - Designed and assembled a 2×4 series-parallel transducer array embedded within an elastomeric silicone damper enclosure beneath an acrylic impact plate.
    - Built and tested a low-loss Schottky bridge rectifier (1N5819), 5.1 V Zener overvoltage clamp, and 0.1 F / 5.5 V supercapacitor storage circuit.
3. **Embedded Firmware Development:**
    - Programmed the ESP32 microcontroller with 16× oversampling and Exponential Moving Average smoothing ($\alpha = 0.25$) on ADC1 channels (GPIO 34 and GPIO 35).
    - Implemented and calibrated a dual-threshold Schmitt trigger with hysteresis ($V_{\text{high}} = 0.80\,\text{V}$, $V_{\text{low}} = 0.55\,\text{V}$) and a 250 ms refractory lockout window, achieving zero false double-triggering.
    - Formulated JSON telemetry serialization and secure HTTPS POST transmission.
4. **Cloud Backend and Database Pipeline:**

- Engineered a RESTful API in Node.js, Express, and TypeScript with SHA-256 device key authentication and Google OAuth 2.0 sessions with HttpOnly cookies.
- Configured MongoDB Atlas cloud database with compound time-series indexes.
- Implemented Socket.IO room namespaces for real-time telemetry streaming.
- Formulated and executed 101 automated Vitest unit, security, and integration tests (100% pass rate).

5. **Machine Learning and Web Dashboard:**

- Trained a Scikit-Learn Random Forest classifier on 5 time-domain kinetic features, validating generalizability via subject-independent Group K-Fold cross-validation (88.6% accuracy).
- Developed a responsive React 19 + TypeScript dashboard featuring live SVG gauges, time-series charts, AI classification feeds, and closed-loop GPIO load toggling.

Project Presentation / Activity Photos & Review Paper Placeholder

Insert photographs of project presentations, project competition participation receipts, patent/copyright filing acknowledgements, or conference review paper acceptance receipts here.

Verified by Project Guide: [GUIDE NAME] · **Department of Information Technology**